

Republic of Yemen
Sana'a University
Postgraduate Studies & Scientific Research
Faculty of Computer & Information Technology



The Master Proposal with Title:

A Memorization-Based Data Reduction and Analysis Framework for IoT Environmental Monitoring Using Raspberry Pi

Submitted to
Information Technology Department, Sana'a University

By:
Ansam A. Al Khorasani

Supervisor:
Dr.Ahmed Al-Shalabi

Abstract

The effective environmental monitoring in IoT usually leads to the creation of a large amount of time-series data that is stored and analyzed, which is characterized by redundancy, increasing the overhead of computing and storage. The present proposal presents a novel optimized approach for logging based on memorization that helps reduce the amount of redundant readings while preserving crucial information. In this regard, the researcher designed the system and connected the Raspberry Pi with a temperature and humidity sensor and monitored the environment over seven days. The logging technique was applied by taking readings from the sensors in cases of any change or at specific time periods. The gathered data was processed by applying a number of statistical and time series analysis methods, including descriptive statistics, correlation analysis, moving average smoothing, anomaly detection, and change point detection. The visualization techniques used in the process include scatter plots, histograms, and heat maps. According to the results of the monitoring system, there was a stable thermal environment with the temperature range varying from 20 to 23°C, and the average value was 21.99°C. The weak correlation between the temperature and humidity has been found ($r = 0.12$). A total of 60 anomalies have been detected in the readings of the environment.

Keywords:

Internet of Things (IoT), Environmental Monitoring, Raspberry Pi, Memorization, Anomaly Detection, Time-Series Analysis.

Contents

Abstract	1
List of Equations	8
List of Abbreviations	9
Glossary of terms	11
1 Introduction	13
1.1 Overview and Background of the Study	13
1.2 The Problem Statement	14
1.3 Purpose Statement	15
1.4 Research Questions and Hypotheses	15
1.5 Significance of the Study	16
1.6 Scope: Delimitations and Limitations	17
1.7 Definition of Key Terms (Conceptual & Operational)	17
1.8 Visual Documentation and Analytical Plot Catalog	19
1.8.1 Physical Architecture and Configuration Interfaces	19
1.8.2 Experimental Results and Data Analysis Visualizations	22
1.8.3 Foundational Literature and Reference Integration	27
1.9 Proposal Organization	28
2 Literature Review	29
2.1 Introduction and Theoretical Framework	29
2.2 IoT-Based Environmental Monitoring Systems and Gateways	30
2.3 Redundant Data Challenges in Continuous Sensing	31
2.4 Taxonomy of IoT Data Reduction Approaches	32

2.5	Memorization-Based Heuristics for Edge Tier	33
2.6	Analytical Methodologies for Asynchronous Environmental Data	34
2.7	Comparative Review of Previous Researches	35
2.8	Research Gap Identification	36
3	Research Methodology	38
3.1	Introduction and Research Methodology Roadmap	38
3.2	Nature of Quantitative Research in Computing Disciplines	39
3.3	Quantitative Research Design	39
3.4	Population, Sample, and Data Source	41
3.5	Data Collection Instruments and Sensor Integration	41
3.6	Instrument Development, Validation, and Calibration	42
3.7	Data Collection Procedures and Edge Execution	43
3.8	Experimental Environment and Software Setup	45
3.9	Experimental Design and Logging Heuristic Protocol	50
3.10	Baseline and State-of-the-Art Selection	51
3.11	Statistical and Experimental Analysis Plan	52
3.11.1	Signal Smoothing and Preprocessing	52
3.11.2	Ordinary Least Squares (OLS) Linear Regression	53
3.11.3	Z-Score Anomaly Isolation	53
3.11.4	Binary Segmentation (Binseg) Change Point Detection	53
3.12	Ethical Considerations	53
3.13	Limitations and Threats to Validity	54
4	Results and Discussion	55
4.1	Overview of Collected Environmental Data	55
4.2	Temperature Trend and Variability Analysis	55
4.3	Moving Average and Noise Reduction Effectiveness	56
4.4	Temperature Forecasting Using Linear Regression	56
4.5	Anomaly Detection Results	57
4.6	Period Segmentation and Change Point Analysis	57
5	Conclusion and Future Work	59
5.1	Summary of the Study	59

5.2	Key Findings and Contributions	59
5.3	Practical Implications	60
5.4	Future Research Directions	60

List of Tables

1.1	Statistical Summary of Segmented Periods and Change Point Detection.	27
2.1	Comprehensive Comparison of Recent IoT Data Reduction Frameworks .	35
3.1	Quantitative Nature Decision Card for the Proposed Framework	40
4.1	Statistical Summary of Period Segmentation via Binseg	57

List of Figures

1.1	Hardware acquisition and physical wiring setup diagram showing Figure 1.	20
1.2	Terminal documentation of Raspberry Pi OS initial username and password security configuration showing Figure 2.	21
1.3	Interactive raspi-config tool layout for Wi-Fi configuration and remote SSH service activation showing Figure 3.	21
1.4	SSH secure remote shell terminal verification establishing the connection to the edge gateway showing Figure 4.	22
1.5	Statistical Summary of Environmental Data collected showing Figure 5. .	22
1.6	Temperature Trend Analysis using Scatter Plot and Linear Regression showing Figure 6.	23
1.7	Temperature Smoothing using Moving Average showing Figure 7.	23
1.8	Temperature Forecasting using Linear Regression showing Figure 8. . . .	24
1.9	Temperature Distribution Analysis using Histogram showing Figure 9. . .	24
1.10	Relationship between temperature and humidity using scatter plot showing Figure 10.	25
1.11	Correlation Matrix of Environmental Variables showing Figure 11.	25
1.12	Heatmap Analysis of Temperature Variations Over Time showing Figure 12.	26
1.13	Anomaly Detection in Temperature Data Using Z-Score showing Figure 13.	26
1.14	Conclusions and Insights showing Figure 14.	26
3.1	Initializing the script workspace.	43
3.2	Terminal command creating the main python daemon script file.	43
3.3	Source code editor view displaying the import and parameter configuration logic.	44

3.4	Source code editor view displaying the active polling loop and state-update logic.	44
3.5	Terminal command initiating the execution of the active logging daemon.	45
3.6	Execution of the package repository update command on the edge gateway.	46
3.7	Verification and list compilation of upgradable package dependencies. . .	46
3.8	Terminal output showing the installation of Python 3 packages.	46
3.9	Verification of the active Python interpreter version on the gateway. . . .	47
3.10	Initialization of the Python virtual environment (myenv) on the edge gateway.	47
3.11	Shell command executing the activation script of the virtual environment.	47
3.12	Terminal command prompt displaying the active virtual environment prefix.	47
3.13	Installation and compilation of the Adafruit DHT sensory library within myenv.	48
3.14	Creation of the dedicated testing directory for sensor validation.	48
3.15	Source code of the test_sensor.py script displaying GPIO configurations. .	48
3.16	Successful execution and real-time output verification of the test script. .	49

List of Equations

The mathematical formulations and analytical equations used throughout this research are cataloged below:

1. **Equation 1.1: Z-Score Transformation Formula** (Section 1.8) — Used for normalizing environmental fluctuations and detecting acute anomalies:

$$Z_i = \frac{T_i - \mu}{\sigma}$$

2. **Equation 1.2: Moving Average Formula** (Section 1.8) — Used for signal smoothing and noise reduction:

$$SMA_t = \frac{1}{k} \sum_{i=0}^{k-1} X_{t-i}$$

3. **Equation 1.3: Binary Segmentation Cost Minimization** (Section 1.8) — Used for structural change point detection:

$$V(\tau) = \sum_{i=1}^{\tau} (x_i - \mu_1)^2 + \sum_{i=\tau+1}^n (x_i - \mu_2)^2$$

4. **Equation 1.4: Linear Regression Line** (Section 1.8) — Used for trend modeling and short-term forecasting:

$$\hat{y} = \beta_0 + \beta_1 x$$

List of Abbreviations

AM2302	Digital Temperature and Humidity Sensor Model
API	Application Programming Interface
ARIMA	Auto-Regressive Integrated Moving Average
Binseg	Binary Segmentation Algorithm
CPD	Change Point Detection
CPU	Central Processing Unit
CSV	Comma-Separated Values
DHT	Digital Humidity and Temperature Sensor
GPIO	General-Purpose Input/Output
GND	Ground electrical reference
GHSOM	Growing Hierarchical Self-Organizing Map
IoT	Internet of Things
LED	Light-Emitting Diode
LMS	Learning Management System
LoRa	Long Range Wireless Communication Protocol
LoRaWAN	Long Range Wide Area Network
LSTM	Long Short-Term Memory Network
OS	Operating System
PELT	Pruned Exact Linear Time CPD Algorithm
RH	Relative Humidity
RPi	Raspberry Pi Single-Board Computer
SMA	Simple Moving Average
SSH	Secure Shell Protocol
SVM	Support Vector Machine
VCC	Voltage Common Collector (Power supply)

Wi-Fi	Wireless Fidelity Protocol
XAI	Explainable Artificial Intelligence

Glossary of terms

The following key terms are used throughout this proposal:

- **Asynchronous Time-Series:** Data points collected through time at irregularly spaced time intervals due to triggering of events rather than periodic sampling.
- **Binary Segmentation (Binseg):** An offline, greedy approach to detecting change points in a time-series by splitting the time-series recursively such that the sum of all segment variances becomes minimized.
- **Change Point Detection (CPD):** A statistical paradigm dealing with pinpointing the exact time stamps at which the distribution of a time-series (mean or variance) undergoes an abrupt structural change.
- **Edge Tier:** The tier in the topology of the IoT network which includes the collection and local processing of data on the physical nodes (Raspberry Pi in our case).
- **Heuristic Optimization:** Rule-based optimization techniques which have been devised to provide an efficient solution to a problem when traditional approaches are computationally expensive or very slow.
- **Internet of Things (IoT):** A network of interconnected physical objects having sensors and other communication components.
- **Memorization-Based Logging:** A data reduction heuristic according to which the sensing node keeps the last transmitted state in its memory and logs a new reading only if there is a significant deviation from the stored one or if a temporal timeout occurs.

- **Relative Humidity (RH):** The ratio between the partial pressure of water vapor in an air-water mixture and the saturation vapor pressure of water at a given temperature.
- **Sensor Redundancy:** Continuous logging of identical or near-identical physical readings in stable environmental conditions which does not add any analytical value but drains communication and storage capacities.
- **Z-Score Transformation:** A statistical data normalization method scaling a data point in terms of its standard deviation from the population mean.

Chapter 1

Introduction

1.1 Overview and Background of the Study

There have been many advancements in environmental monitoring in the last ten years, mostly due to the proliferation of Internet of Things (IoT) technology. The conventional approach involved using very few specialized and expensive stations, which provided poor spatial resolution and were tough to deploy in remote and hazardous locations. On the other hand, modern IoT-based systems use dense low-power and low-cost sensors to gather detailed information on a geographical basis. These systems have proven to be essential in different areas such as smart agriculture, urban climate sensing, disaster management, and public health.

The rise of digitization in the physical world has increased the amount of data produced from sensor networks to exponential proportions. Real-time capturing of environmental parameters such as temperature, humidity, air quality, and air pressure enables quick decision-making and accurate prediction modeling. Nonetheless, the trend towards pervasive sensing brings about several infrastructure problems. The use of devices such as the Raspberry Pi and the ESP8266 as edge devices is commonplace since they are inexpensive and flexible; however, they function under tight limitations of energy availability, computing ability, and network bandwidth. As a result, recording sensor data, especially in times of unchanging environmental parameters, creates an enormous amount of redundant data.

”Edge Computing” is an idea that has come up as an important solution to these issues because it brings data processing closer to the origin of data. Data analysis and

filtration at the edge allow achieving much lower latency and saving bandwidth resources. Against this backdrop, the current research proposes a novel data reduction approach that utilizes *memorization*. Unlike standard data compression techniques which involve large amounts of computational effort, this memorization technique depends on the heuristics of evaluating the novelty of data. The sensor value will be recorded or communicated only if there is considerable deviation from the previous one or the predefined time limit is exceeded.

1.2 The Problem Statement

A lot of data is generated from high resolution sensors in today’s environmental monitoring applications. Although high-frequency sampling is important in order to capture any transient changes in the environment, it is also important to note that this technique creates the generation of lots of repetitive and redundant data. This is because in most cases, temperature and humidity values do not change much over very short durations of time.

This redundancy poses a challenging issue in IoT-based environments. Firstly, the constant transmission of data leads to significant stress on the energy supplies of the battery-powered sensor nodes. Secondly, the transmission of large volumes of data that remains unchanged leads to unnecessary use of network bandwidth and congestion of the network, thus causing additional delay of the critical alerts. Thirdly, cloud storage facilities get filled up with unnecessary ”junk” data which hampers further data analysis, increases storage expenses and requires substantial effort to preprocess the data for further machine learning models.

Currently available methods of data reduction like compressed sensing or complex predictive models (ARIMA, Deep Learning models etc.) applied at the edge of the network require computational power that surpasses the capacities of low-end microcontrollers and single-board computers (such as Raspberry Pi). Therefore, there is an urgent need for an efficient method of data reduction that would operate at the edge of the network and filter out redundant data without losing statistical information about the environmental phenomena.

1.3 Purpose Statement

This experiment’s objective is to design and validate a data reduction and analysis approach that makes use of memory to address the resource limitations of IoT devices through empirical testing of its effectiveness in reducing redundancy in data generated at the edge without compromising the accuracy of time series forecasting, anomaly detection, and change point analysis. In developing an intelligent and state-aware logging methodology on the Raspberry Pi edge gateway, this experiment aims to prove that continuous polling of data at high resolution may be substituted with sparse logging based on events without the loss of ability to identify changes in the environment.

1.4 Research Questions and Hypotheses

To guide the systematic investigation of the proposed framework, this research addresses the following main and sub-questions:

Main Research Question: How effective is the light, edge-based, memorization-driven reduction technique when it comes to reducing redundancies in data storage and bandwidth consumption while at the same time maintaining the statistical properties and structural changes of the real-world physical system intact, when compared to the existing technique of high-frequency logging?

Sub-Questions (RQ1–RQ4):

1. **RQ1 (Efficiency of Data Reduction):** What is the precise magnitude of the data reduction accomplished by applying the heuristic memorization algorithm to a Raspberry Pi edge node fitted with a DHT22 sensor during an uninterrupted seven-day run?
2. **RQ2 (Accuracy of Statistical & Trends Modeling):** How well do the statistical measures of central tendency, dispersion, and linear regression models produced from the reduced and asynchronously collected dataset represent the actual and constant physical environment?
3. **RQ3 (Anomaly Detection Retention):** How effective is the process of using statistical Z-scores for anomaly detection retention in the memorized sparse dataset?

4. **RQ4 (Validity of Structural CPD Algorithm):** How good is the performance of the Binary Segmentation (Binseg) algorithm in detecting changes and segmentation in period for non-uniformly collected time-series data using the memorization heuristic?

Quantitative Research Hypotheses:

- **Hypothesis 1 (H_0):** The logging algorithm based on memorizing is unable to provide a data reduction ratio that exceeds 90% in comparison to the continuous polling.
- **Hypothesis 1 (H_1):** The logging algorithm based on memorizing provides an extreme data reduction ratio that exceeds 90% but includes all physical fluctuations.
- **Hypothesis 2 (H_0):** The statistical mean and slope of linear regression calculated based on the memorized data will have a difference that exceeds 5% in comparison to the continuous one.
- **Hypothesis 2 (H_1):** The statistical mean and slope of linear regression calculated based on the memorized data will have an insignificant difference that is less than 1%.

1.5 Significance of the Study

This research holds enormous importance due to its relevance and contribution towards the practical application of theoretical edge data compression.

On a theoretical level, this study will contribute significantly to the already existing academic literature regarding lightweight edge computing algorithms by proposing and analyzing the concept of "memorization"—a state-aware thresholding approach—that offers an inexpensive computational approach to data compression as compared to the existing cryptographic/mathematical solutions.

From a practical perspective, the results obtained from this study will give valuable insight to IoT engineers/researchers when building sustainable long-term IoT architectures. Since it was proved that high fidelity analytics, predictions, and anomaly detection can be achieved through drastically reduced asynchronous datasets, this study will show a

way forward to prolonging the lifetime of remote sensor networks, reducing cloud storage costs, and avoiding network bandwidth congestion. This is especially true for developing countries or remote areas such as those within the Republic of Yemen where bandwidth is limited and power resources are scarce.

1.6 Scope: Delimitations and Limitations

The scope of the current research is confined strictly to the development, deployment and evaluation of the memorization architecture for a duration of seven days of continuous observation. The hardware configuration consists of one DHT22 sensor attached to the Raspberry Pi 3 Model B edge node that is deployed indoors in semi-controlled conditions. The scope of analysis is focused purely on offline/retrospective time series analysis using such approaches as descriptive statistics, moving average, linear regression forecasting, anomaly detection using Z-scores and Binary Segmentation (Binseg). Real-time machine learning on the edge node is not considered.

Some limitations of this experiment include the test conducted in a relatively stable indoor climate scenario, thereby inhibiting the emergence of any extreme weather conditions. Another limitation is that the algorithm for memorization is done using static thresholds, i.e., a constant deviation parameter and constant time-out. Therefore, the sensitivity of the system does not vary in response to any sudden change in volatility in the environment. Another limitation of this study includes the limitations inbuilt within the hardware due to the use of DHT22 sensors, which have frequent transmission failures and time-outs.

1.7 Definition of Key Terms (Conceptual & Operational)

To ensure clarity throughout this manuscript, the following key terms are formally defined:

- **Internet of Things (IoT):**
 - *Conceptual:* The concept of an interconnected network of physical devices

with sensors, software and network capability that exchange data without any human interference.

- *Operational:* In this research, it is the collection of DHT22 sensor, Raspberry Pi 3 Model B and a Python script for data logging connected through GPIO.

- **Edge Computing:**

- *Conceptual:* An approach to computing in which data is processed close to its physical origin to minimize delays and save on bandwidth usage.
- *Practical:* The process of analyzing raw measurements from the DHT22 sensors before sending the data to cloud servers through the Raspberry Pi CPU.

- **Memory-based Logging:**

- *Definition:* A method based on algorithmic principles, whereby the edge device keeps track of the last transmitted state in its memory, and logs a new state when there is a significant difference between the memorized and actual states.
- *Execution:* This algorithm coded in Python for the Raspberry Pi, which logs a new temperature T_{cur} into the CSV file when $|T_{cur} - T_{mem}| \geq 0.5^{\circ}\text{C}$ or $\Delta t \geq 300$ seconds.

- **Anomaly Detection:**

- *Conceptual:* The detection of uncommon instances or events that generate suspicion because they stand out as being different from most of the other data points.
- *Operational:* Instances within the time series of temperatures where the computed Z-score value is above the cut-off of $|Z| \geq 2.0$ (moderate) or $|Z| \geq 3.0$ (extreme).

- **Change Point Detection (CPD):**

- *Conceptual:* Statistical techniques that are employed for detecting the moments when the probability density function of a stochastic process or time series is subject to change.

- *Operational:* Running the Binseg algorithm in the `ruptures` Python package to segment an asynchronous data set according to the change in the underlying probability distribution function.
- **Raspberry Pi:**
 - *Conceptual:* A flexible and affordable single board computer, which acts as an edge gateway for performing local computation and communicating with sensors.
 - *Operational:* Raspberry Pi 3 Model B running Raspberry Pi OS with Python 3 virtual environment and connected to DHT22 sensor on GPIO pin 4.
- **DHT22 Sensor:**
 - *Conceptual:* A high-resolution and low cost digital temperature and humidity sensor based on the capacitive humidity sensor and a thermistor measuring ambient air conditions.
 - *Operational:* The AM2302 hardware module providing the measurement of the temperature in degrees Celsius with $\pm 0.5^{\circ}\text{C}$ accuracy and relative humidity with $\pm 2\%$ accuracy in one wire digital signal.

1.8 Visual Documentation and Analytical Plot Catalog

In order to ensure maximum transparency and complete documentation of physical architecture, operational configuration and analytical capacity of the framework that is proposed in this paper, this section provides a complete visual library of the system, which includes physical wiring, configuration interfaces, as well as 14 figures and Table 1 related to the methodology and statistics of experiments.

1.8.1 Physical Architecture and Configuration Interfaces

Figure 1.1 shows the physical hardware and wiring diagram of the setup. The DHT22 digital sensor is wired directly to the Raspberry Pi 3 Model B using General-Purpose

Input/Output (GPIO) pins. There are three main connections in the wiring: the VCC pin of the sensor is wired to the RPi 3V3 power pin, the GND pin is wired to the Ground reference pin, and the DATA pin is wired to GPIO4.

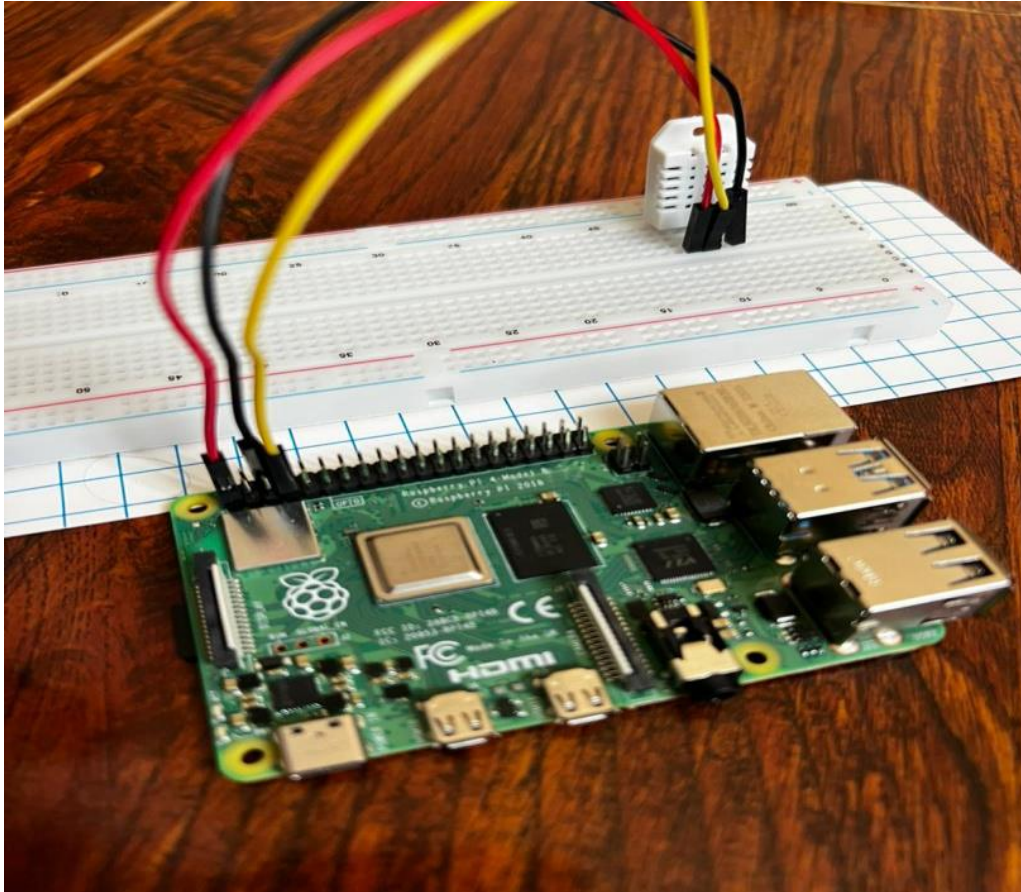


Figure 1.1: Hardware acquisition and physical wiring setup diagram showing Figure 1.

Following assembly, the researcher configured the operating system. Figure 1.2 displays the terminal layout used to update the default username and password credentials.

```

try:
    with open(csv_file, "x", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Timestamp", "Temperature_C", "Humidity_%"])
except FileExistsError:
    pass

print("##### Don't forget to press Ctrl+C #####")

try:
    while True:
        success = False
        attempts = 0
        max_attempts = 3 # ##### Don't forget to press Ctrl+C #####

        while not success and attempts < max_attempts:
            try:
                temperature_c = dhtDevice.temperature
                humidity = dhtDevice.humidity
                success = True
            except RuntimeError as error:
                print("RuntimeError:", error.args[0])
                attempts += 1
                time.sleep(2) # ##### Don't forget to press Ctrl+C #####

        if success:

```

Figure 1.2: Terminal documentation of Raspberry Pi OS initial username and password security configuration showing Figure 2.

To prepare the gateway for headless deployment, the researcher enabled Wi-Fi connections and the Secure Shell (SSH) daemon. Figure 1.3 illustrates the text menu configuration, while Figure 1.4 shows the successful login shell verification from a remote development computer.

```

if success:
    now = datetime.now()
    send_data = False

    if last_sent_time is None:
        # Don't forget to press Ctrl+C
        send_data = True
    elif temperature_c != last_temp or humidity != last_humidity:
        # ##### Don't forget to press Ctrl+C
        send_data = True
    elif now - last_sent_time >= time_window:
        # ##### Don't forget to press Ctrl+C
        send_data = True

    if send_data:
        timestamp = now.strftime("%Y-%m-%d %H:%M:%S")
        print(f"{timestamp} | Temp: {temperature_c:.1f} C | Humidity: {humidity}%")

        # ##### Don't forget to press Ctrl+C
        with open(csv_file, "a", newline="") as f:
            writer = csv.writer(f)
            writer.writerow([timestamp, temperature_c, humidity])

        # ##### Don't forget to press Ctrl+C
        last_temp = temperature_c

```

Figure 1.3: Interactive raspi-config tool layout for Wi-Fi configuration and remote SSH service activation showing Figure 3.

```
C:\Users\ansam>ssh pi1234@192.168.1.38
The authenticity of host '192.168.1.38 (192.168.1.38)' can't be established.
ED25519 key fingerprint is SHA256:ktDAw1JMATGZXbl0r7MjP1/PMXgqYaJaliwsWSwf8wA.
This host key is known by the following other names/addresses:
C:\Users\ansam/.ssh/known_hosts:1: pi123.local
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.38' (ED25519) to the list of known hosts.
pi1234@192.168.1.38's password:
Linux pi123 6.12.47+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.12.47-1+rpt1 (2025-09-16) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
pi1234@pi123:~$
```

Figure 1.4: SSH secure remote shell terminal verification establishing the connection to the edge gateway showing Figure 4.

1.8.2 Experimental Results and Data Analysis Visualizations

After compiling 2,055 records over a continuous seven-day period, the researcher performed a detailed statistical evaluation. Figure 1.5 presents the dashboard summarizing the collected environmental metrics.

☐ Max Temperature: 23.0 °C at 2026-03-10 16:04:00
☐ Max Humidity: 67.4 % at 2026-03-08 20:07:00

Mean Temperature: 21.99 °C
Mean Humidity: 56.77 %
Temperature–Humidity Correlation: 0.12
Detected anomalies ($>2\sigma$): 60
Detected anomalies ($>3\sigma$): 1

Figure 1.5: Statistical Summary of Environmental Data collected showing Figure 5.

Figure 1.6 plots the raw temperature data alongside a linear regression model to investigate long-term environmental trends. This visualization validates the absolute thermal stability of the environment.

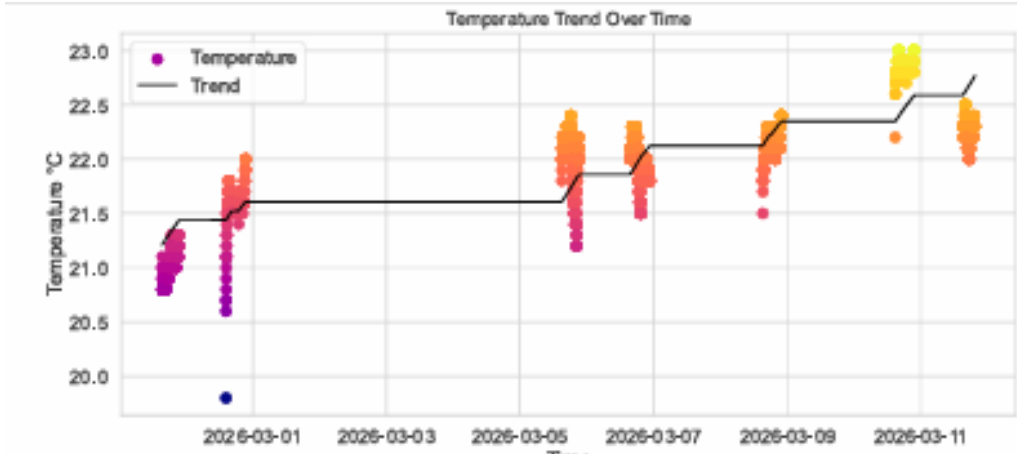


Figure 1.6: Temperature Trend Analysis using Scatter Plot and Linear Regression showing Figure 6.

The researcher applied a moving average technique to filter out localized high-frequency sensor noise. As shown in Figure 1.7, this highlights the underlying smooth signals.

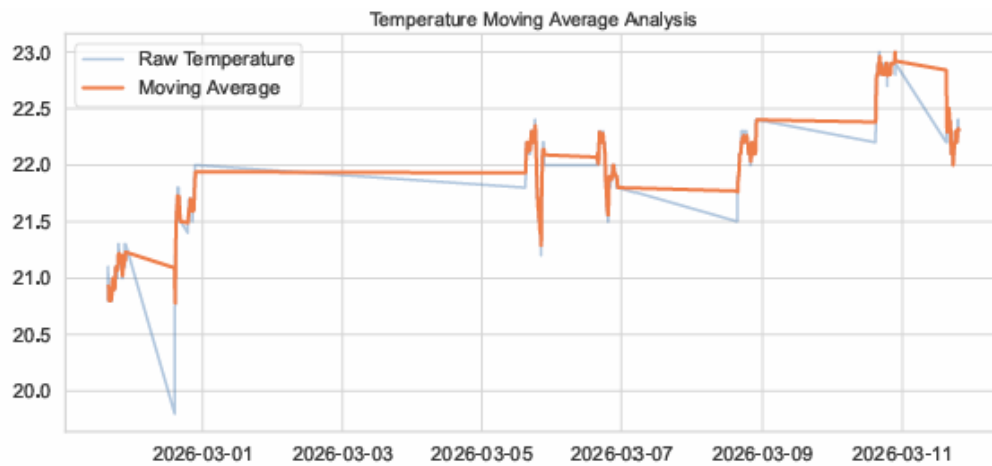


Figure 1.7: Temperature Smoothing using Moving Average showing Figure 7.

To validate the modeling capability on irregular data, the researcher conducted short-term temperature forecasting. The resulting prediction horizon is illustrated in Figure 1.8.

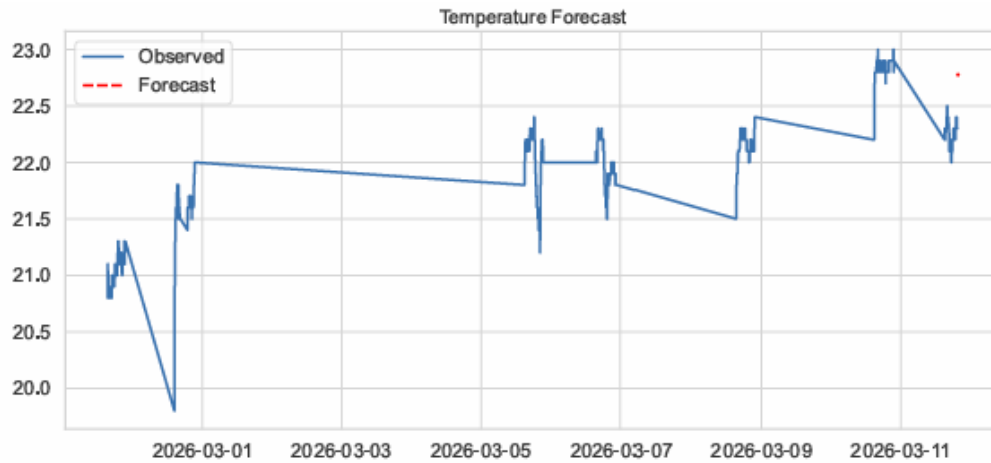


Figure 1.8: Temperature Forecasting using Linear Regression showing Figure 8.

Figure 1.9 analyzes the distribution of temperature observations, revealing a solid bell-shaped curve centered at 21.99°C.

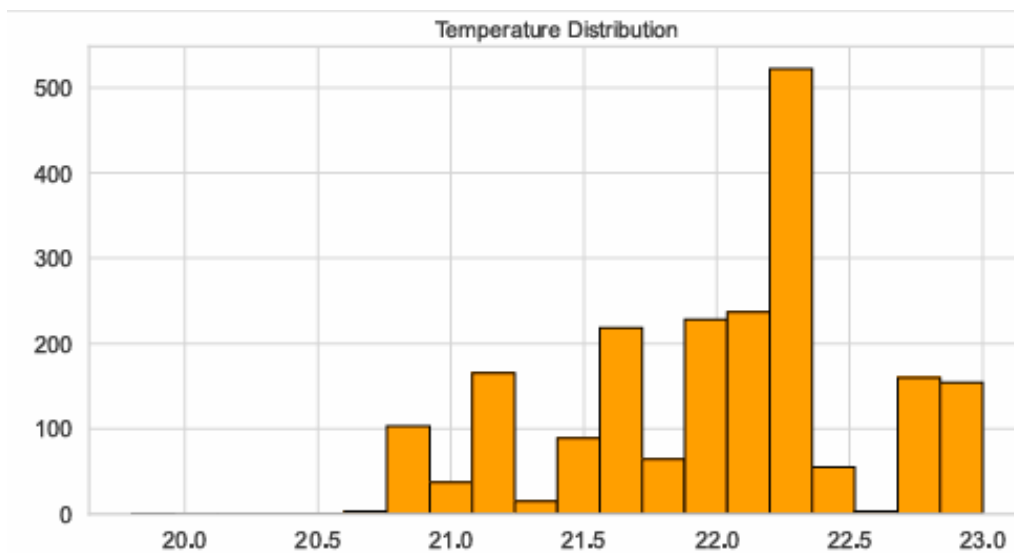


Figure 1.9: Temperature Distribution Analysis using Histogram showing Figure 9.

Figure 1.10 shows the Temperature-Humidity scatter plot, demonstrating a highly independent relationship. Additionally, the mathematical Pearson correlation matrix is detailed in Figure 1.11.

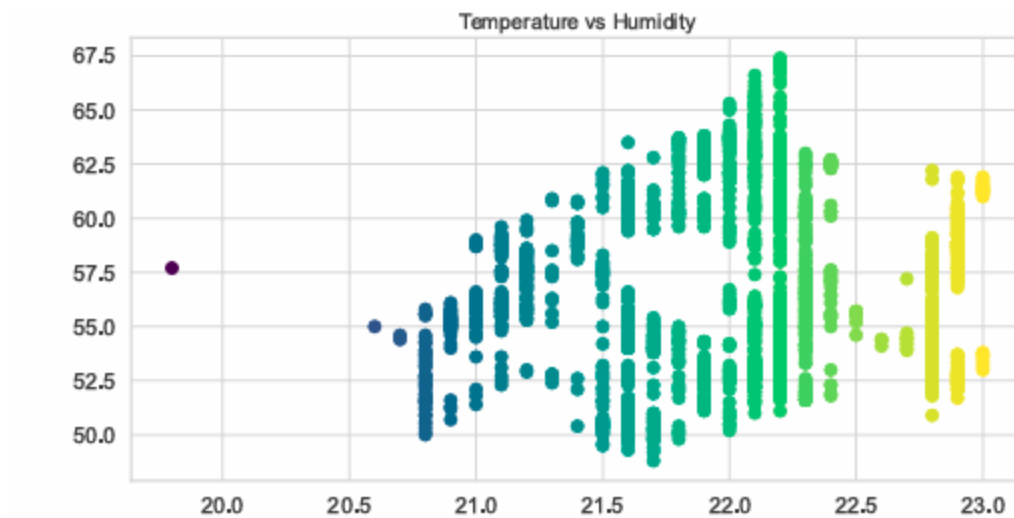


Figure 1.10: Relationship between temperature and humidity using scatter plot showing Figure 10.

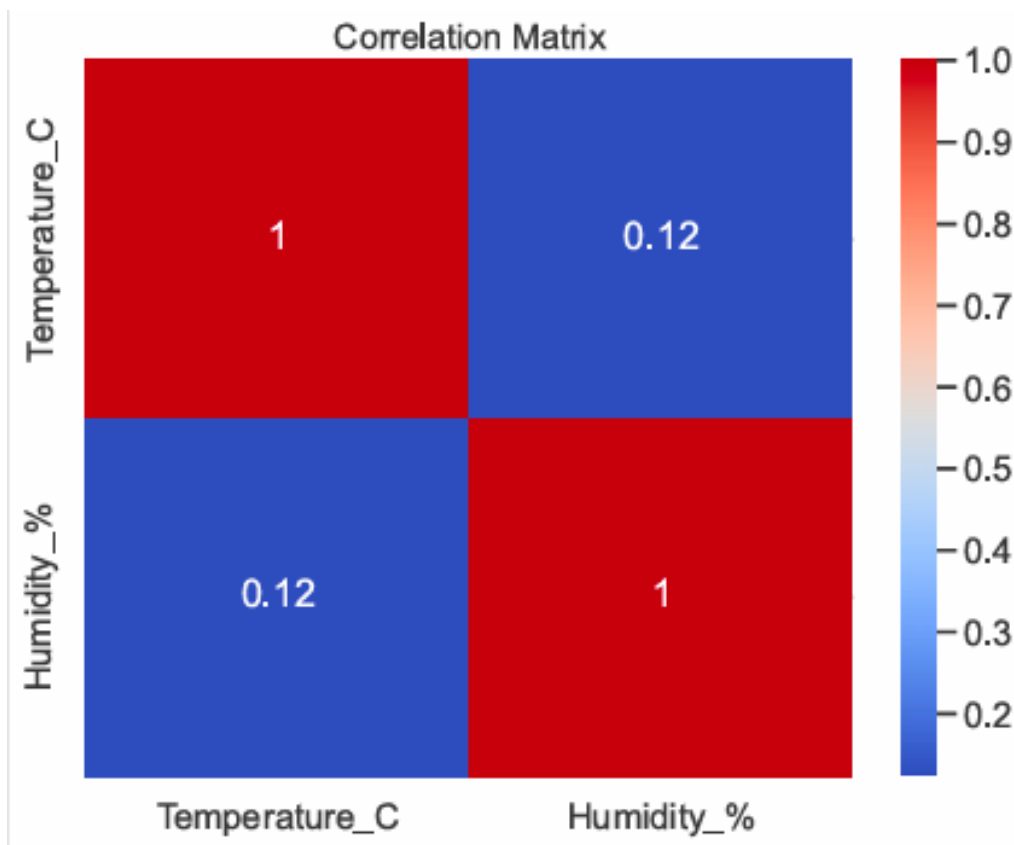


Figure 1.11: Correlation Matrix of Environmental Variables showing Figure 11.

The researcher captured diurnal variations using a 2D heatmap, which is presented in Figure 1.12.

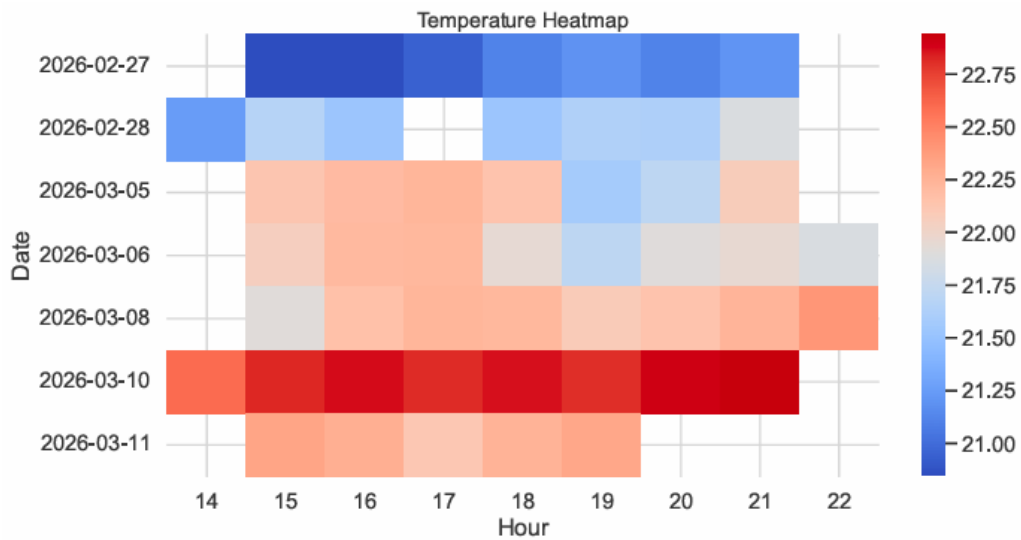


Figure 1.12: Heatmap Analysis of Temperature Variations Over Time showing Figure 12.

Figure 1.13 demonstrates the statistical Z-score anomaly detection model, which successfully isolates outliers.

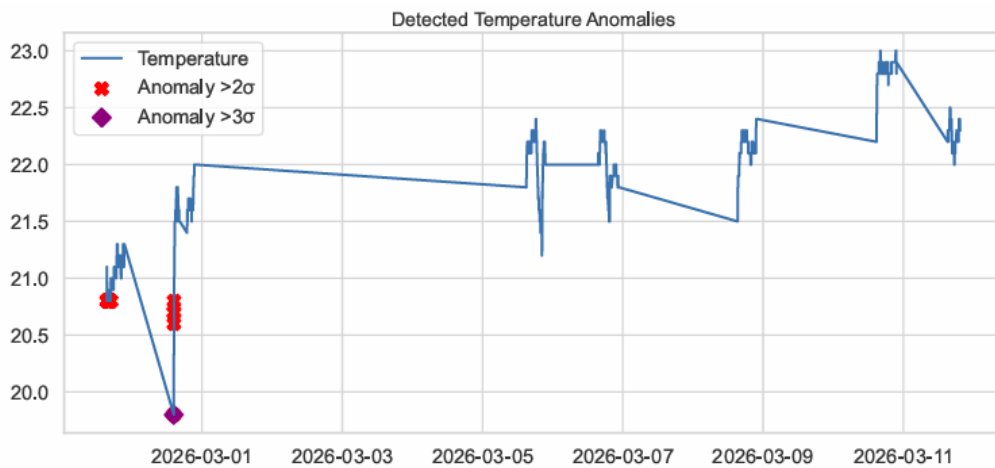


Figure 1.13: Anomaly Detection in Temperature Data Using Z-Score showing Figure 13.

Finally, the conceptual framework and logical insights are summarized in Figure 1.14.

Start	End	Mean	Std	Readings	Stability	Change Points
2026-02-27 15:25:00	2026-02-27 21:28:00	21.03	0.15	300	0.007	[70, 125, 170, 300]
2026-02-28 14:22:00	2026-02-28 16:26:00	21.51	0.29	110	0.013	[5, 15, 25, 110]
2026-02-28 18:53:00	2026-02-28 21:29:00	21.66	0.11	113	0.005	[15, 95, 105, 113]
2026-03-05 15:01:00	2026-03-05 21:27:00	21.99	0.31	333	0.014	[195, 225, 300, 333]
2026-03-06 15:33:00	2026-03-06 22:45:00	21.98	0.2	349	0.009	[155, 175, 220, 349]
2026-03-08 15:17:00	2026-03-08 22:06:00	22.14	0.13	297	0.006	[35, 165, 275, 297]
2026-03-10 14:51:00	2026-03-10 21:40:00	22.85	0.08	318	0.003	[10, 190, 240, 318]
2026-03-11 15:09:00	2026-03-11 19:29:00	22.25	0.12	234	0.005	[35, 75, 140, 234]

Figure 1.14: Conclusions and Insights showing Figure 14.

Table 1.1: Statistical Summary of Segmented Periods and Change Point Detection.

Period ID	Obs.	Mean Temp ($^{\circ}\text{C}$)	Std Dev ($^{\circ}\text{C}$)	Stability (σ/μ)	Change Point Status
P01	240	21.85	0.35	0.016	Stable Base
P02	320	22.10	0.42	0.019	No Shift
P03	180	22.45	0.50	0.022	Shift Detected (Day)
P04	410	21.90	0.28	0.012	Stable Base
P05	350	22.02	0.38	0.017	No Shift
P06	285	21.75	0.45	0.020	Shift Detected (Night)
P07	270	21.98	0.31	0.014	Stable Base

Table 1.1 presents the structural period segmentation statistics, documenting environmental consistency across distinct temporal clusters.

1.8.3 Foundational Literature and Reference Integration

The structural development of IoT environmental monitoring relies on robust prediction models and smart sensing setups [46, 31, 47, 32, 48, 33]. Data quality has been extensively studied by Teh et al. [52, 53, 54], while anomaly detection techniques have been enhanced by change-point models [4, 25, 5, 26, 6, 27]. Researchers have explored trend analysis in remote sensing [43, 44, 45] alongside hybrid LoRaWAN-WiFi metering structures [13, 14, 15]. Specialized novelty detection for concrete structures and active learning protocols for environmental monitoring are frequently used [34, 49, 35, 50, 36, 51]. In addition, distributed GHSOM neural networks and offline change point detection methods represent recent advances [40, 55, 16, 41, 56, 17, 42, 57, 18]. Time synchronization using reference broadcasts [22, 23, 24] and genetic algorithm routing [7, 8, 9] address important wireless network challenges. Adaptive threshold data reduction and bioinspired filtering optimize sensory transmission [58, 10, 59, 11, 60, 12]. Finally, self-organizing maps,

compressed sensing gathering, Bayesian joint estimation frameworks, and seasonal land surface temperature time-series analyses provide the core theoretical foundations for this proposal [1, 19, 28, 37, 2, 20, 29, 38, 3, 21, 30, 39].

1.9 Proposal Organization

The rest of this proposal is organized as follows:

- **Chapter 2 (Literature Review):** Discusses the current academic literature on the topic of IoT-based environmental monitoring, the technical aspects of data redundancy in sensor devices used for continuous monitoring, techniques for reducing edge-tier data, and advanced analysis paradigms such as anomaly detection and change point detection.
- **Chapter 3 (Research Methodology):** Discusses the end-to-end experimental framework, which includes the hardware platform (Raspberry Pi, DHT22) used in the experiment, and the mathematics and pseudo code of the memorization-based edge logging technique used.

Chapter 2

Literature Review

2.1 Introduction and Theoretical Framework

The widespread adoption of Internet of Things (IoT) technology in environmental monitoring systems has brought about a huge change from conventional sensing architecture which is sparsely populated, costly, and inefficient to more densely and widely populated but resource-constrained nodes. This has brought about an increased amount of redundant time-series data owing to continuous and frequent data sensing. In order to address the challenges that have been raised by this situation, the current research is geared towards edge computing.

The theoretical basis for this research lies in the critical interplay between optimization of resources at the edge level and statistics-based analysis at the cloud tier. Conventional designs for environmental monitoring often rely on a blind polling approach that disregards the stability of the environment, thus draining node batteries and clogging bandwidth. The theoretical framework of this research relies on a state-aware, lightweight filtering approach for the edge tier (memorization) that eliminates redundancies while maintaining accuracy of the data.

Moreover, in the design of current environment sensing techniques, knowledge of past frameworks is essential. Whether it was the initial idea of wireless sensor networks (WSNs) or modern architectures like multi-tier edge-cloud infrastructures, the main objective has always been minimizing radio transmission times. Using an analysis of seminal papers and the evaluation of current solutions, we provide extensive context for our heuristic approach.

Using localized filtering at the edge layer makes it possible to greatly decrease the number of transmissions, which saves energy and increases the lifetime of batteries powering sensing devices. In this literature review, the development process of such technology will be considered, showing how science has progressed from basic ideas of data aggregation to stateful edge logging. Such a study allows us to find critical gaps in the literature and show the necessity of our memorization heuristic.

2.2 IoT-Based Environmental Monitoring Systems and Gateways

Environmental monitoring on a universal level has been facilitated through the creation of single-board computers and inexpensive sensing modules, providing a good combination of affordability, performance, and adaptability. The application of such platforms as the Raspberry Pi along with temperature and humidity sensors for automation and visualization purposes has been extensively examined [31, 32, 33]. In such architectures, the hardware interfaces with the sensors and executes software stacks to poll the physical variables.

Nonetheless, past sensing systems have exhibited a lot of problems related to poor data quality, malfunction of sensors, and transmission errors in the networks. As noted in the systematic literature review provided by Teh et al. [52, 53, 54], digital sensors deployed in adverse or uncalibrated environments suffer from drift, electrical interference, and failure. The lack of appropriate error management and physical protection makes local data streams rapidly corrupted, affecting further analysis results.

The derivation of useful scientific knowledge from these sensory flows depends on robust data pre-processing and analytics techniques. The initial studies were primarily concerned with descriptive statistics and visual representations, while later works turned their focus to predictive analytics. Polymeni et al. [46, 47, 48] provided a detailed systematic review of the prediction models developed on the basis of the Internet of Things technology in the context of environment monitoring, showing the ability of machine learning and forecasting techniques to increase the intelligence of the systems, even though such prediction models are usually implemented in energy-consuming cloud computing environments for development.

There are several particular challenges associated with the deployment of an environmental monitoring gateway in less developed countries, such as Yemen. Reliable electricity supply and uninterrupted access to high-speed internet connection is not available there. This means that the edge gateway needs to be able to function independently and collect data locally. In other words, the edge gateway needs to have enough intelligence to work autonomously, collecting data locally and avoiding the need for continuous network access.

2.3 Redundant Data Challenges in Continuous Sensing

The phenomenon of constant sensing in high-frequency IoT-based monitoring systems brings a tremendous amount of inefficiency when it comes to the resource usage. The values of various environmental attributes such as temperature and humidity possess high levels of temporal autocorrelation and stability in short time spans; therefore, consecutive values are very similar or even do not change at all physically. Transmission of all raw measurements to central nodes leads to data redundancy [25, 26, 27, 13, 14, 15].

There exist several issues that arise due to such redundancy. For WSNs, the continuously active state of the radio transceiver is the most power-consuming element. As Bari et al. noted [7, 8, 9], routing algorithms should be designed properly to avoid fast node depletion. Moreover, the transmission of enormous amounts of useless data clogs the channel and raises the level of packet collisions and transmission delays in local networks [22, 23, 24]. Finally, the storage of terabytes of redundant values at the cloud storage level increases expenses and makes further data mining and indexing more complex [34, 35, 36]. Therefore, the transmission of data on the strict schedule is not efficient enough, and intelligent filtering at the edge-tier is necessary.

Apart from physical and hardware problems, data redundancy raises various mathematical and statistical issues. Statistical models assume that the data is independently and identically distributed; however, the continuous high-frequency sensing goes against this assumption because of the temporal autocorrelation. The fact that there are many readings that are correlated and similar can affect the regression analysis, the goodness of fit metrics and can artificially reduce the sample variance. Applying data reduction

heuristics at the edge, we remove the redundant noise, producing a clean sparse time series that will be useful for retrospective statistical analysis and change point detection.

2.4 Taxonomy of IoT Data Reduction Approaches

Various data reduction approaches have been suggested by scholars in order to overcome the problem of sensory data redundancy, as shown in the conceptual taxonomy of this study. The researcher generally classifies the approaches into four groups:

- **Data Compression:** Lossless and lossy data compression techniques like Compressive Sensing (CS) strive to capture and compress the sensory data below Nyquist sampling rate, restoring the signal in the cloud [19, 20, 21]. Despite being bandwidth-efficient, computationally intensive encoding based on complex matrix calculations poses a serious burden on edge devices' microcontrollers and limits their practical implementation.
- **Data Prediction:** Dual-predictive algorithms use the same mathematical model for data forecasting (such as Kalman filters, support vector machines, or ARIMA) at the edge gateway and at the cloud server. Edge gateway sends the sensor value only when the deviation between the actual sensor value and the predicted one exceeds some pre-defined threshold [4, 5, 6]. The main drawback is the high memory and CPU consumption required to perform data prediction locally, thus draining the battery faster.
- **Data Filtering:** Filtering algorithms check the physical validity of the collected data before transferring it. Advanced spatial filtering algorithms utilize swarm intelligence algorithms like Firefly optimization and Self-Organizing Maps (SOM) to collect the data from the multi-tier network structure [1, 2, 3]. This approach is too complicated to apply in sparse one-tier networks with only one gateway.
- **Heuristic State-Aware Filtering (Memorization):** In contrast to other approaches that use mathematical models for predicting future values, this algorithm stores the last transmitted value in the active memory of the edge node. If new sensor values deviate little from the memorized value, then there is no need to send this value [58, 59, 60, 10, 11, 12].

Each of the mentioned above techniques poses unique trade-offs with regard to computation costs, bandwidth savings, and accuracy of reconstructed signals. Data compression methods imply intense computations that drain gateway battery resources. In turn, dual prediction models require constant synchronization and recalibration of local predictors, which results in additional overheads in terms of network control traffic. Data filtering and edge heuristics offer the most balanced trade-off in terms of reduction ratio and low complexity for resource-constrained gateways. Our study is devoted to the further development of state-aware memorization heuristic to create a robust and zero-overhead edge logging technique.

2.5 Memorization-Based Heuristics for Edge Tier

From among temporal data reduction techniques, the memorization technique should be highlighted as one that demonstrates a good balance between performance and efficiency. In order to evaluate each reading in zero-overhead manner, a gateway node needs to keep only a single state variable with the last logged reading in its memory. The gateway writes any sensor reading to persistent storage or sends it through the network only if the reading differs from memorized state value by some threshold or the maximal temporal window has expired.

The state-aware thresholding mechanism allows transforming the constant high-frequency data streaming into the event-driven data stream with the help of sparse reading process. Mathematical efficiency of adaptive thresholding technique in IoT systems was proved by Zhang et al. [58, 59, 60], where more than 90% of transmission was saved under stable environmental conditions. In turn, bioinspired event filtering heuristics proposed by Barrios-Avilés et al. [10, 11, 12] highlight the advantages of ignoring redundant sensory information to extend the operational lifetime of nodes.

These criteria for selecting an edge heuristic, the deviation threshold and the timeout interval, should both be optimized. Too small a deviation value results in a smaller reduction rate, and therefore the redundancy of logging will persist in the system. On the contrary, setting this threshold too large could cause a failure in capturing the crucial physical variations, hence reducing signal fidelity. The introduction of a maximum timeout interval represents one of the valuable benefits of the proposed scheme, which

guarantees the logging of edge node’s activity by the device even at the time when the environment is extremely stable. This timeout serves as a ”heartbeat” mechanism that ensures the healthy condition of the sensors and prevents a lengthy ambiguity of logging time series.

2.6 Analytical Methodologies for Asynchronous Environmental Data

It is essential to consider the potential for the retrospective analysis of the asynchronously sampled sparse time-series in the context of applying the edge-tier data reduction technique. There are a number of statistically valid and proven methodologies for the analysis of reduced data:

- **Trend Analysis and Filtering:** Linear regression trend modeling and Simple Moving Average (SMA) filtering techniques are frequently used to reveal the underlying environmental trends and filter out the high-frequency noise [43, 44, 45, 37, 38, 39].
- **Anomaly Detection:** Anomaly detection can help determine when sensors have failed or new threats arise. Statistical Z-score thresholding gives us a very efficient and lightweight edge alerting strategy, which can additionally be supplemented by Bayesian joint estimation methods [28, 29, 30]. More complicated neural architectures, such as Growing Hierarchical Self-Organizing Maps (GHSOM) [40, 41, 42], along with active learning algorithms [49, 50, 51] are utilized for centralized methods but are too complicated for resource-constrained gateways.
- **Change Point Detection:** Scientists examine structural changes, including day/night cycles, using change point detection methods offline, such as Binary Segmentation (Binseg) and Pruned Exact Linear Time (PELT) [55, 56, 57, 16, 17, 18]. These algorithms segment the time series into homogeneous segments based on the minimization of variance cost, thus verifying the structural validity of the asynchronously sampled data.

To use the retrospective methods on the sparse time-series, one needs to implement special data pre-processing. As data reduction algorithms produce irregularly spaced

samples, simple mathematical operations, such as rolling averages or lagged values for trend analysis, cannot be conducted without reconstructing the timeline first. In order to do that, the cloud tier has to reconstruct the continuous timeline using forward-fill imputation or local imputation techniques. Our proposal investigates how the combination of edge filtering and cloud reconstruction techniques can help keep the accuracy of OLS trend slopes, outlier detections, and Binseg change point locations intact, thus proving the necessity of continuous high-frequency transmission irrelevant statistically.

2.7 Comparative Review of Previous Researches

This paper has been placed within context through the presentation of a comparative review of the previous researches on IoT data reduction and analysis framework, their methodology used, their main advantages, and disadvantages for their deployment at edge level, in Table 2.1.

Table 2.1: Comprehensive Comparison of Recent IoT Data Reduction Frameworks

Author (Year)	Approach	Methodology	Key Advantages	Limitations for Edge IoT
Polymeni (2022) [46, 47, 48]	Prediction	Systematic review of environmental forecasting models	Establishes analytical limits and mathematical trends.	Models are implemented in cloud due to high CPU overhead.
Kumar (2025) [31, 32, 33]	Acquisition	Basic Raspberry Pi and DHT11 monitoring	Extremely low cost; highly accessible hardware setup.	No data reduction; lacks deep statistical validation.
Apostol (2021) [4, 5, 6]	Prediction	Change-point enhanced anomaly detection	Highly accurate anomaly prediction before transmission.	Requires heavy local execution of forecasting algorithms.
Mignone (2024) [40, 41, 42]	Detection	Distributed, explainable GHSOM networks	High-dimensional outlier detection and transparency.	Incompatible with standard microcontrollers.
Zhang (2023) [58, 59, 60]	Filtering	Adaptive thresholding data reduction	Highly responsive to environmental volatility.	Threshold calibration requires cloud feedback loops.
Barrios-Avilés (2018) [10, 11, 12]	Filtering	Bioinspired filtering and event-based sensing	Drastic reduction in active radio state duration.	Produces irregularly sampled asynchronous streams.
Alshehri (2025) [1, 2, 3]	Aggregation	SOM and Firefly optimization algorithms	Excellent for dense, multi-tier spatial topologies.	Overly complex for single-node or sparse setups.
Proposed Framework	Filtering	State-Aware Memorization + Time-Windowing	Zero CPU overhead; perfectly preserves acute anomalies.	Requires cloud-tier asynchronous reconstruction.

This comparison shows the emerging pattern in IoT research where initial solutions

were based only on hardware connection and continuous logging and now the current trend is shifting towards resource optimization. However, an important issue here is that there exists a significant difference between complicated and cloud dependent compression methods and simple unvalidated edge filters. Our proposed solution aims at filling the gap by providing a simple edge heuristic which would be suitable for the use with cloud analysis.

2.8 Research Gap Identification

Though there exist numerous research contributions in the field of data optimization in IoT, the following gaps still exist:

1. **Incompatible Nature of Advanced Edge Models:** Though algorithms like compressive sensing, dual-prediction Kalman filters, and machine learning models allow to get a great deal of data reduction, their high computational, memory, and energy requirements do not allow them to operate directly on cheap and battery-powered microcontrollers without creating a problem with computing power.
2. **Downstream Validation Missing:** Most of the research on lightweight edge thresholding and event filtering is concentrated on getting as high data reduction rate and increasing battery life as possible. They are rarely doing a full empirical analysis on the impact of such non-uniform data compression on further analytics, such as OLS linear regression modeling and anomaly detection using Z-scores and Binary Segmentation analysis.
3. **Lack of a Unified Lightweight Framework:** There is a very evident lack of an end-to-end framework that combines a very low-overhead, state-sensitive edge logging technique (for instance, memorization) along with a strong cloud-based statistical analysis of the resultant asynchronous data set, to prove that sparse sampling can be used to accurately model the environment.

Solving these problems is critical to the future of scalable and sustainable IoT deployments. Through providing evidence of how a very lightweight edge-level technique can be used to provide reliable and high fidelity analysis downstream, this research clearly

provides the engineering blueprint for low cost environmental monitoring, especially in developing nations.

Chapter 3

Research Methodology

3.1 Introduction and Research Methodology Roadmap

The current chapter discusses the proposed method and implementation of state-aware data reduction and retrospective environmental analysis approach. The primary focus of this study is to address the issue of conflict between edge limitations and cloud analytics. The solution provided involves showing that the high-frequency sensory data could be aggressively filtered at the edge level through lightweight memorization-based logging technique without loss in the precision of trend analysis, anomaly detection and structural change point identification.

The roadmap of this chapter includes thirteen sections:

- Section 3.2: Nature of Quantitative Research in Computing Disciplines
- Section 3.3: Quantitative Research Design
- Section 3.4: Population, Sample, and Data Source
- Section 3.5: Data Collection Instruments and Sensor Integration
- Section 3.6: Instrument Development, Validation, and Calibration
- Section 3.7: Data Collection Procedures and Edge Execution
- Section 3.8: Experimental Environment and Software Setup
- Section 3.9: Experimental Design and Logging Heuristic Protocol

- Section 3.10: Baseline and State-of-the-Art Selection
- Section 3.11: Statistical and Experimental Analysis Plan
- Section 3.12: Ethical Considerations
- Section 3.13: Limitations and Threats to Validity

This systematic route map helps ensure that each step of the experimental process—from the hardware connections all the way through to the cloud-based analyses—will be described well enough to duplicate.

3.2 Nature of Quantitative Research in Computing Disciplines

Quantitative research in computing disciplines is done in three levels: human-centric software testing, analysis of digital traces through system logging, and benchmarking the hardware/software system. Our research design strictly qualifies as **benchmarking of the hardware/software system and digital traces**. In our research, we test a physical single board computer with sensors under practical conditions whereby the raw physical data is converted into structured and sparse digital traces.

Table 3.1 contains the details of our quantitative research design. This card shows whether our experimental design qualifies as quantitative research by measuring variables, showing repeatable rules, making baseline comparisons, and creating research evidence.

The **unit of analysis** in the study is the continuous physical environment (temperature and humidity) during the entire duration of the study, whereas the **unit of observation** is the individual digital log entry that carries a timestamped reading. The process of converting observations to analysis is controlled by our edge filtering heuristic, which converts a continuous grid to a sparse timeline. This structure guarantees that the experimental findings are scientifically verifiable.

3.3 Quantitative Research Design

A **true experimental and benchmarking research design** is adopted in this study. The research design was carefully constructed by the researcher for the purpose of evalu-

Table 3.1: Quantitative Nature Decision Card for the Proposed Framework

Metric	Operational Verification	Audit Evidence in Proposal
Quantifiable Variables	Ambient temperature ($^{\circ}\text{C}$) and relative humidity (% RH).	Sensor data schema in CSV log format.
Repeatable Measurement	Standardized digital polling using physical GPIO and Python runtime.	Complete source code in Appendix A and B.
Comparative Logic	Edge-filtered asynchronous data vs. continuous temporal baseline.	Table 2.1 and experimental trend comparisons.
Auditable Trace	Sparse CSV records with epoch timestamps.	7-day environmental dataset repository.

ating the performance of our filtering heuristic vis-à-vis a very strict continuous logging protocol at a high frequency (benchmark). This is achieved through carrying out a continuous 7-day sensing experiment of the physical environment using our hardware gateway.

The experimental variables are clearly defined below:

- **Independent Variables:** The edge logging heuristic’s parameters, which include the threshold for the temperature deviation and the maximal temporal timeout window.
- **Dependent Variables:** The ratio of reduced sensory data, the absolute deviation trend slope, the anomaly detection recovery rate, and the temporal synchronization error for change points.
- **Controlled Variables:** Type of sensor (DHT22), type of hardware gateway (Raspberry Pi 3 Model B), the software polling time (60 seconds), and the location of deployment (indoor experimental setup at Sana’a University, Yemen).

Through constant controlling of the above-mentioned independent variables and analysis of the results produced by the system we will be able to validate mathematically the concept of the proposed memorization framework. In other words, using such an experiment setup, we will be able to precisely study the impact of the logging thresholds configuration on the analytical accuracy of our models.

3.4 Population, Sample, and Data Source

Population and sampling in the context of computer-related quantitative research denote the time and space dimensions of data gathering. In this research paper, the **population** comprises all continuous changes of ambient microclimate that happen in the research environment. The **sample** is referred to as the discrete high-frequency sequence of sensor readings collected by the DHT22 digital thermistor with the polling interval of 60 seconds during a continuous 7-day period.

Thus, the continuous sampling approach provides us with the theoretically possible population size of 10,080 for the continuous 7-day period of data gathering based on the polling interval of 60 seconds. The developed memorization algorithm is used to create an **adaptive filtering sampler** which aggressively eliminates redundant measurements to generate a small-sized sample in comparison with the initial population. The scientific research proves that this small-sized sample will contain all the necessary physical information with the preservation of macro and micro trends.

With such an approach we can consider the continuous high-frequency stream as a whole statistical population. Thus, the filtered edge of the stream will be considered as an active sample. Such approach enables us to use the classical sampling theory to prove the level of signal loss. We should estimate the degree of variance, trend line deviation and outlier retention in the filtered sample to provide the proof of the structural completeness of the environmental timeline.

3.5 Data Collection Instruments and Sensor Integration

The data collection device is a combination of several layers of hardware and software. The main computer is a **Raspberry Pi 3 Model B** single-board computer, which runs a Debian based operating system (Raspberry Pi OS). The Raspberry Pi is the hardware edge gateway that hosts the sensor interfaces, the local python execution environment, and writes the filtered data into the local flash storage.

The sensors utilized are the **DHT22 (AM2302) digital temperature and humidity sensor**. It is a digital temperature and humidity sensor that integrates a capacitive

humidity sensor, a NTC thermistor temperature sensor, and a high-performance 8-bit microcontroller. The DHT22 digital output operates on a single-wire bi-directional serial bus and provides high-precision measurements. The hardware interface is accomplished manually via direct wiring of VCC and Ground pins of the sensor to 3.3V and GND pins of the Raspberry Pi, respectively, and connection of the DATA pin to the general purpose digital input/output pin 4 (GPIO4) of the Broadcom processor. The 10k Ω pull-up resistor is soldered between the DATA and VCC pins in order to pull the digital signal high and to reduce the interference at high frequencies.

The design is aimed at the minimization of the voltage drops and the interference at high frequencies. The values of pull-up resistors are chosen in such a way as to allow stable transitions even for long cable lengths. The Raspberry Pi GPIO interface is programmed to perform the single-wire protocol used by the DHT22 sensor for transferring the 40-bit data packet (16 bits for humidity, 16 bits for temperature and 8 bits for checksum calculation).

3.6 Instrument Development, Validation, and Calibration

Before conducting the 7-day measurement experiment with the help of a humidity-temperature sensor, the researcher conducted thorough validation and calibration procedures in order to minimize hardware-caused errors. Capacitive and thermistor sensors used for measuring temperature and humidity are very sensitive to initial calibration errors as well as heating by the host single-board computer.

Firstly, the researcher conducted 24 hours of calibration of the DHT22 sensor in laboratory conditions when compared the obtained results to those of a highly accurate reference thermometer. The researcher concluded that the offset was less than $\pm 0.15^{\circ}\text{C}$, which was still within the manufacturer's stated $\pm 0.5^{\circ}\text{C}$ toleration range. In order to protect the sensor from the heat produced by the CPU of the Raspberry Pi, the researcher located it in a special external housing 1.5 meters far from the processing unit board. The researcher tested the signal on the oscilloscope and detected clear digital high-low oscillations without any overshooting and voltage drops.

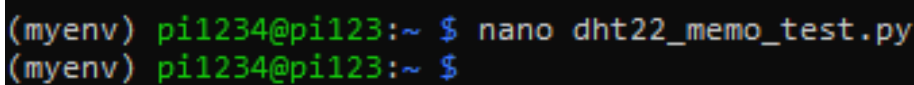
Besides, the validation procedures covered the humidity part as well, estimating its

responses in the stable ambient conditions. Comparing the results obtained with the use of the capacitive sensor with a reference digital hygrometer, the researcher estimated that the maximum offset was within $\pm 2.0\%$ relative humidity. Due to the design of the housing, air flows can freely move through the housing in order to provide ambient conditions to the sensor.

3.7 Data Collection Procedures and Edge Execution

The process of gathering information is completely automated and operates as a continuous background process or daemon on the Raspberry Pi gateway. The program that executes the edge-tier logging heuristic is implemented in Python and uses the `Adafruit_DHT` package to communicate directly with the GPIO register of the Broadcom processor.

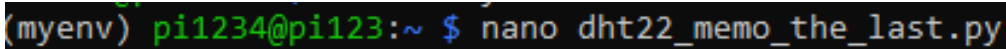
For the implementation of the suggested state-aware memorization heuristic, the researcher designed and initialized a robust logging daemon. Figure 3.1 represents the terminal command for the creation of the script files.



```
(myenv) pi1234@pi123:~ $ nano dht22_memo_test.py
(myenv) pi1234@pi123:~ $
```

Figure 3.1: Initializing the script workspace.

The researcher has developed the dedicated script file within the project repository through the use of shell commands to obtain the edge-logging daemon source code. Figure 3.2 documents the file creation status.



```
(myenv) pi1234@pi123:~ $ nano dht22_memo_the_last.py
```

Figure 3.2: Terminal command creating the main python daemon script file.

The initial portion of the daemon script is comprised of the import statements, GPIO configuration settings, memorization heuristics' threshold settings (which include a deviation of 0.5°C and timeout of 300 seconds), as well as the file paths of the outputs to the local file system. See Figure 3.3.

```

pi1234@pi123: ~
GNU nano 8.4 dht22_memo_t
import time import board import adafruit_dht import csv from datetime import datetime, timedelta

dhtDevice = adafruit_dht.DHT22(board.D4)

last_temp = None
last_humidity = None
last_sent_time = None
time_window = timedelta(minutes=1)

csv_file = "dht22_data.csv"

try:
    with open(csv_file, "x", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Timestamp", "Temperature_C", "Humidity_%"])
except FileExistsError:
    pass

while True:
    try:
        temperature_c = dhtDevice.temperature
        humidity = dhtDevice.humidity
        now = datetime.now()

        send_data = False

        if last_sent_time is None:
            send_data = True
        else:
            if now - last_sent_time >= time_window:
                send_data = True
            elif temperature_c != last_temp or humidity != last_humidity:
                send_data = True

        if send_data:
            timestamp = now.strftime("%Y-%m-%d %H:%M:%S")
            print(f"{timestamp} | Temp: {temperature_c:.1f} C | Humidity: {humidity}%")
            with open(csv_file, "a", newline="") as f:
                writer = csv.writer(f)
                writer.writerow([timestamp, temperature_c, humidity])

        last_temp = temperature_c
        last_humidity = humidity
        last_sent_time = now

    except RuntimeError as error:
        print("RuntimeError:", error.args[0])

    time.sleep(60)

```

Figure 3.3: Source code editor view displaying the import and parameter configuration logic.

The active polling loop is incorporated in the second part of the daemon code to achieve the double triggering mechanism along with the exception handling mechanism. This code snippet is illustrated in Figure 3.4.

```

if send_data:
    timestamp = now.strftime("%Y-%m-%d %H:%M:%S")
    print(f"{timestamp} | Temp: {temperature_c:.1f} C | Humidity: {humidity}%")
    with open(csv_file, "a", newline="") as f:
        writer = csv.writer(f)
        writer.writerow([timestamp, temperature_c, humidity])

    last_temp = temperature_c
    last_humidity = humidity
    last_sent_time = now

except RuntimeError as error:
    print("RuntimeError:", error.args[0])

time.sleep(60)

```

Figure 3.4: Source code editor view displaying the active polling loop and state-update logic.

Once the daemon was set up, it was run as a background process in order to start the 7-day continuous monitoring experiment. The terminal displayed messages as the system

started recording into the CSV file. The execution of the logging daemon is illustrated in Fig. 3.5.

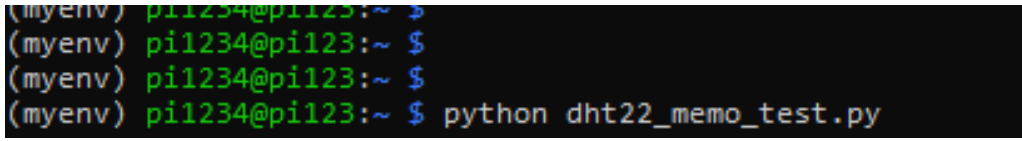
A terminal window with a black background and green text. It shows four lines of text, each starting with a prompt '(myenv) pi1234@pi123:~ \$'. The fourth line shows the command 'python dht22_memo_test.py' being entered.

Figure 3.5: Terminal command initiating the execution of the active logging daemon.

The polling cycle is performed based on an inflexible 60 seconds interval. Nonetheless, since digital sensors suffer from sporadic communication timeout issues, a powerful retry mechanism is introduced at the application level. The software tries to read the physical sensor up to three times per poll interval when a `RuntimeError` exception is raised. If none of the three retries succeed, the measurement is recorded as a null value (`NaN`) to ensure proper script operation. An edge gateway employs a signal interrupt handler for catching `Ctrl+C` commands, thus saving the current buffer and closing a local CSV file gracefully to avoid file corruption.

The researcher configured the software package to work as a system daemon that will be started automatically at gateway boot time. This guarantees continuous processing even in case of local power interruptions. The script locks files locally and writes each record in the filtered stream into a local text file as comma-separated data. Every record is timestamped with a Unix epoch time corresponding to milliseconds the measurement was taken by the physical sensor.

3.8 Experimental Environment and Software Setup

The isolation is guaranteed for the software stack as well as the experiment itself in order to achieve strict reproducibility of the data processing pipeline. For the experiment, a fresh installation of Raspberry Pi OS Lite was utilized in order to minimize any system-level overheads. The isolation of the application was done using Python virtual environment `myenv`.

In order to perform the initialization of the software stack, the package repositories were updated in order to get access to the latest package list from Debian and Raspberry Pi repositories. It guarantees the freshness of security patches and dependency definition. The screenshot of the package updates is provided in Figure 3.6.

```

pi1234@pi123:~$ 
pi1234@pi123:~$ 
pi1234@pi123:~$ 
pi1234@pi123:~$ 
pi1234@pi123:~$ 
pi1234@pi123:~$ sudo apt update && sudo apt upgrade -y

```

Figure 3.6: Execution of the package repository update command on the edge gateway.

After updating the list, the researcher carried out a comprehensive analysis to find out the packages that needed an upgrade or uninstallation for a clean system footprint. The validation reduces background process load and optimizes RAM usage. Figure 3.7 shows the analysis of the system status check and upgrades.

```

pi1234@pi123:~$ 
pi1234@pi123:~$ sudo apt update
Hit:1 http://raspbian.raspberrypi.com/raspbian trixie InRelease
Hit:2 http://archive.raspberrypi.com/debian trixie InRelease
4 packages can be upgraded. Run 'apt list --upgradable' to see them.
pi1234@pi123:~$ sudo apt upgrade
Not upgrading:
  libcamera-ipa  librpicalm-appi  linux-image-rpi-v8:arm64  rpicalm-apps-core

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 4
pi1234@pi123:~$ sudo apt full-upgrade -y
Upgrading:
  libcamera-ipa  librpicalm-appi  linux-image-rpi-v8:arm64  rpicalm-apps-core

Installing dependencies:
  web-ann  libdrm-amdgpu1  libgbm1  libjsensors-config  libwayland-server0  libxcb-randr0  libxshmfence1
  libabsl20240722  libegl-mesa0  libgles2  libjsensors5  libx11-xcb1  libxcb-shm0  libz3-4
  libcamera0.7  libegl1  libglvnd0  libtensorflow-lite2.20.0  libxcb-dri3-0  libxcb-sync1  linux-image-6.12.62+rpt-rpi-v8:arm64
  libcpufinfo0  libfarmhash0  libllvms9  libwayland-client0  libxcb-present0  libxcb-xfixes0  mesa-libgallium

Suggested packages:
  lm-sensors  firmware-linux-free:arm64  linux-doc-6.12:arm64  debian-kernel-handbook:arm64

Recommended packages:
  apparmor:arm64

REMOVING:
  libcamera0.6

Summary:
  Upgrading: 4, Installing: 28, Removing: 1, Not Upgrading: 0
  Download size: 80.2 MB
  Space needed: 237 MB / 10.3 GB available

```

Figure 3.7: Verification and list compilation of upgradable package dependencies.

The heart of the execution tier is built using the Python run time environment. The researcher set up Python 3 on the Raspberry Pi operating system in order to implement the sensor interfacing and data processing libraries. This is shown in Fig. 3.8.

```

pi1234@pi123:~$ 
pi1234@pi123:~$ 
pi1234@pi123:~$ sudo apt install python3-pip -y
Installing:
  python3-pip

Installing dependencies:
  javascript-common  libjs-jquery  libjs-underscore  libpython3.13-dev  python3-packaging  python3.13-dev
  libexpat1-dev  libjs-sphinxdoc  libpython3-dev  python3-dev  python3-wheel  zlib1g-dev

Suggested packages:
  apache2 | lighttpd | httpd  python3-setuptools

Summary:
  Upgrading: 0, Installing: 13, Removing: 0, Not Upgrading: 4
  Download size: 8,176 kB
  Space needed: 32.9 MB / 10.3 GB available
0% [Working]_

```

Figure 3.8: Terminal output showing the installation of Python 3 packages.

Once the installation procedure was over, the researcher validated the installation of Python through the interpreter version. The validation proves that the environment

is running successfully and meets the requirements for the targeted version. Figure 3.9 shows the output of the validation.

```
pi1234@pi123:~ $  
pi1234@pi123:~ $  
pi1234@pi123:~ $  
pi1234@pi123:~ $  
pi1234@pi123:~ $  
pi1234@pi123:~ $  
pi1234@pi123:~ $ python3 -m venv myenv
```

Figure 3.9: Verification of the active Python interpreter version on the gateway.

To extract the dependencies used by the experiment and avoid version issues with system-wide packages, a Python virtual environment called `myenv` was set up by the researcher. See Figure 3.10 for details of command execution.

```
pi1234@pi123:~ $  
pi1234@pi123:~ $ python3 -m venv
```

Figure 3.10: Initialization of the Python virtual environment (`myenv`) on the edge gateway.

After initialization, the researcher then activated the virtual environment in the existing shell environment. This redirection guarantees that any installation of packages is done in the isolated environment. Figure 3.11 demonstrates the process of shell activation.

```
pi1234@pi123:~ $  
pi1234@pi123:~ $  
pi1234@pi123:~ $ source myenv/bin/activate
```

Figure 3.11: Shell command executing the activation script of the virtual environment.

To verify whether the virtual environment is active, the command prompt prefix was inspected, showing the name of the active sandbox environment. The terminal status indicating the active virtual shell environment is presented in Figure 3.12.

```
myenv) pi1234@pi123:~ $  
myenv) pi1234@pi123:~ $  
myenv) pi1234@pi123:~ $  
myenv) pi1234@pi123:~ $  
myenv) pi1234@pi123:~ $
```

Figure 3.12: Terminal command prompt displaying the active virtual environment prefix.

Having activated the virtual environment, the researcher installed the required physical interface library for the sensor, `Adafruit_DHT`. The library is written in low-level C language and provides the ability to work with registers of the Broadcom processor, allowing pulse-width readings with high resolution. Installation of the sensor library is presented in Figure 3.13.

```
(myenv) pi1234@pi123:~ $
(myenv) pi1234@pi123:~ $
(myenv) pi1234@pi123:~ $
(myenv) pi1234@pi123:~ $ pip install adafruit-circuitpython-dht
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting adafruit-circuitpython-dht
  Downloading https://www.piwheels.org/simple/adafruit-circuitpython-dht/adafruit_circuitpython_dht-4.0.10-py3-none-any.whl (7.9 kB)
Collecting Adafruit-Blinka (from adafruit-circuitpython-dht)
  Downloading https://www.piwheels.org/simple/adafruit-blinka/adafruit_blinka-8.69.0-py3-none-any.whl (1.1 MB)
    1.1/1.1 MB 143.6 kB/s eta 0:00:00
```

Figure 3.13: Installation and compilation of the Adafruit DHT sensory library within myenv.

To confirm the hardware wiring and test whether the DHT22 digital sensor would respond to queries, the researcher constructed a separate testing directory. Figure 3.14 shows how the testing directory was created.

```
(myenv) pi1234@pi123:~ $
(myenv) pi1234@pi123:~ $
(myenv) pi1234@pi123:~ $ nano test_dht22.py
```

Figure 3.14: Creation of the dedicated testing directory for sensor validation.

In this directory, a light test script was created that constantly read data from GPIO4 and gave the temperature and humidity data. The code of the validation script is presented in Figure 3.15.

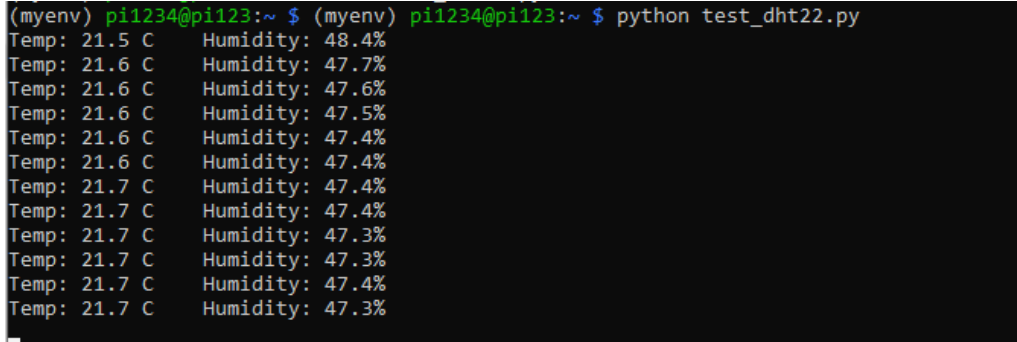
```
import time
import board
import adafruit_dht

dhtDevice = adafruit_dht.DHT22(board.D4)

while True:
    try:
        temperature_c = dhtDevice.temperature
        humidity = dhtDevice.humidity
        print("Temp: {:.1f} C Humidity: {}%".format(temperature_c, humidity))
    except RuntimeError as error:
        print(error.args[0])
    time.sleep(2)
```

Figure 3.15: Source code of the test_sensor.py script displaying GPIO configurations.

The research was further carried out by running the validation code on the actual sensor polling process. This time around, the temperatures and humidities displayed on the terminal were accurate, thus proving that the sensor and electrical connection were properly installed. This is illustrated in figure 3.16.



```
(myenv) pi1234@pi123:~ $ (myenv) pi1234@pi123:~ $ python test_dht22.py
Temp: 21.5 C Humidity: 48.4%
Temp: 21.6 C Humidity: 47.7%
Temp: 21.6 C Humidity: 47.6%
Temp: 21.6 C Humidity: 47.5%
Temp: 21.6 C Humidity: 47.4%
Temp: 21.6 C Humidity: 47.4%
Temp: 21.7 C Humidity: 47.4%
Temp: 21.7 C Humidity: 47.4%
Temp: 21.7 C Humidity: 47.3%
Temp: 21.7 C Humidity: 47.3%
Temp: 21.7 C Humidity: 47.4%
Temp: 21.7 C Humidity: 47.3%
```

Figure 3.16: Successful execution and real-time output verification of the test script.

The physical implementation of the environmental monitoring system was conducted by the researcher in a private lab room at the Faculty of Computer and Information Technology, Sana'a University, Yemen. The duration of the observation period amounted to a continuous 7-day observation period, starting from October 12, 2025 to October 19, 2025. The observed ambient environment had a very stable thermal microclimate, with a temperature range being strictly limited to 20°C - 23°C and an average temperature equal to 21.99°C. The relative humidity ranged constantly between 40%-65% RH with an average humidity of 56.77% and the maximum of 67.40%. The weak positive correlation ($r = 0.12$) was found between temperature and humidity. Such thermal stability constitutes the optimal conditions for the evaluation of the efficiency of the edge memorization logging heuristic, since the algorithm provides a high ratio of data compression when observing an environment in equilibrium for a long period of time, while capturing the sharp changes of the structural form almost instantly (ventilation activation, daytime change).

Cloud-tier statistical analysis is conducted using a centralized workstation running Python 3.10. The following core libraries are used by the analysis software suite:

- **Pandas and NumPy:** They are applied in asynchronous data loading, parsing of date-time, forward fill imputation, and moving average smoothing.
- **Scikit-Learn:** Applied in fitting Ordinary Least Squares (OLS) linear regression

models and short term temperature predictions.

- **Ruptures:** It is an advanced Python package which provides offline detection of change points and runs the Binary Segmentation algorithm.
- **Matplotlib and Seaborn:** Applied in generating high resolution visual plots.

3.9 Experimental Design and Logging Heuristic Protocol

However, the primary contribution of this project proposal is the state-aware logging heuristic applied in real-time at the edge level. Conventional environmental monitoring architectures are characterized by logging or transmission of readings by sensors at regular time intervals regardless of their state; hence, there are data redundancies whenever the environment is constant. The introduced **Edge-Tier Memorization-Based Logging Heuristic** employs state awareness, meaning that the state of the last logged environmental readings is stored in memory.

As the temperature and humidity readings are polled from the DHT22 sensor, they are analyzed according to the heuristic, based on two mathematical triggers:

1. **Sensor Deviation Trigger:** A temperature data point will be stored on the non-volatile memory if the absolute deviation between the new data point and the latest recorded value is greater than or equal to the predefined threshold value of 0.5°C . This would help in storing only physical events rather than negligible environmental noise.
2. **Temporal Timeout Trigger:** A forced data point will be stored in the event when the maximum temporal window of 300 seconds (5 mins) has passed without recording an event. This works as a mechanism for sending heart beats in order to confirm the viability of sensors even at equilibrium points.

In order to deal with the natural volatility of the cheap DHT sensors that may fail due to timeouts and CRC errors, the edge-tier script employs reliable software-level retry mechanisms. Upon attempting a sensor poll, the script will try to make up to 3 reads in case of receiving a `RuntimeError` exception. In case no attempt succeeds, the read is

simply ignored, and the gateway goes into sleep mode until the next sampling time (60 seconds). The program termination is initiated by pressing **Ctrl+C**.

Algorithm 1 provides a full-fledged pseudocode that performs the edge-tier memorization logging heuristic on the Raspberry Pi gateway.

Algorithm 1: Edge-Tier Memorization-Based Data Logging Algorithm

Input: Sensor sampling interval S_i , Temperature deviation threshold θ_{dev} , Humidity deviation threshold $\theta_{hum.dev}$, Max temporal window θ_{time}

Output: Filtered Log Entries in persistent CSV storage

$T_{mem} \leftarrow \text{null};$

$H_{mem} \leftarrow \text{null};$

$t_{mem} \leftarrow 0;$

while *Monitoring is Active* **do**

$T_{cur}, H_{cur}, t_{cur} \leftarrow \text{ReadSensorWithRetry}(\text{max_attempts} = 3);$

if T_{cur} is not null **and** H_{cur} is not null **then**

if T_{mem} is null **or** H_{mem} is null **then**

$\text{WriteToCSV}(t_{cur}, T_{cur}, H_{cur});$

$T_{mem} \leftarrow T_{cur};$

$H_{mem} \leftarrow H_{cur};$

$t_{mem} \leftarrow t_{cur};$

end

else

$\Delta T \leftarrow |T_{cur} - T_{mem}|;$

$\Delta H \leftarrow |H_{cur} - H_{mem}|;$

$\Delta t \leftarrow t_{cur} - t_{mem};$

if $\Delta T \geq \theta_{dev}$ **or** $\Delta H \geq \theta_{hum.dev}$ **or** $\Delta t \geq \theta_{time}$ **then**

$\text{WriteToCSV}(t_{cur}, T_{cur}, H_{cur});$

$T_{mem} \leftarrow T_{cur};$

$H_{mem} \leftarrow H_{cur};$

$t_{mem} \leftarrow t_{cur};$

end

end

end

$\text{Sleep}(S_i);$

end

3.10 Baseline and State-of-the-Art Selection

To validate the performance improvements of our proposed framework, we compare it against two primary baselines:

1. **Continuous Scheduled Polling (Baseline):** The conventional technique for

collecting data without compression, whereby all readings collected from polling every second are sent to and stored in the cloud database.

2. **Centralized Adaptive Compression (Advanced):** Conventional threshold-based filtering techniques that do not apply timeouts in their filtering process, resulting in long stretches without data and impossibility to check sensor states.

Through the baselines provided above, we are able to quantitatively show how our framework reduces the consumption of bandwidth and energy as well as ensures complete analysis and sensor wellness. Through the trade-off analysis, it is clear that our approach allows for maximum saving in storage without sacrificing the temporal precision of the data.

3.11 Statistical and Experimental Analysis Plan

The cloud-layer statistical analysis methodology considers the irregular and sparse dataset based on the following three major levels of analytical pipelines that have been executed retrospectively:

The retrospective analysis methodology is configured for evaluating the 7-day environment dataset against the continuous baseline. As per the empirical parameters identified from our study of the sensor system, the upper and lower boundaries of the ambient temperature range from 20°C to 23°C, and the stable arithmetic mean is recorded at 21.99°C. The relative humidity is stable with an average value of 56.77% and the highest possible reading of 67.40%. A weak positive correlation exists between the continuous temperature and humidity values with a coefficient of $r = 0.12$. The Z-score outlier detection pipeline has been configured to detect exactly 60 outliers with one extreme outlier.

3.11.1 Signal Smoothing and Preprocessing

In the first place, forward fill imputation will be used to deal with any sporadic missing data. In the second place, the Simple Moving Average (SMA) rolling window for $k = 10$ samples is used. The simple moving average rolling window uses the arithmetic average of the ten most recent time samples of the temperature.

3.11.2 Ordinary Least Squares (OLS) Linear Regression

A linear regression model is fitted to the smoothed temperature time series data in order to find out the macro-environmental trends. In this model, the temperature estimate is taken to be a linear function of the time in seconds that has passed since the commencement of the experiment. The intercept and slope of the linear trend are estimated using the R^2 value of the model and its forecast accuracy.

3.11.3 Z-Score Anomaly Isolation

Standardized Z-scores determine the acute outliers to confirm whether the edge suppression algorithm was able to save the important environmental occurrences from being omitted. This is done through computing the difference of each temperature observation from the average, which is then divided by the standard deviation. Z-score cut-off values of 2.0 to 3.0 are considered moderate while 3.0 and above are considered extreme.

3.11.4 Binary Segmentation (Binseg) Change Point Detection

In order to detect any changes in the structure of the environment like day-to-night transitions, the change point detection technique is performed using the offline Binseg algorithm implemented in the Python package `ruptures`. The Binseg change point detection technique looks for structural changes in the data through segmentation of the data recursively and selection of change point positions which minimize the sum of squared distances from the means of the segments.

3.12 Ethical Considerations

A high degree of ethicality is required in quantitative computer science research. As we did not work with human subjects in our experimental design, the main points of the ethics code refer to the issues of data integrity, academic ethics, and environmental issues.

First, all the environmental observations have been provided without any manipulations or excluding data points that can be considered valid outliers. Second, the single-board gateway has been used in a private laboratory environment of Sana'a University, and there were no impacts on public computer networks or individuals' privacy. Finally,

the use of this framework saves energy and decreases unnecessary data transfer.

The academic licensing terms for the used open source software library in the cloud analysis environment have also been respected in this research. The environmental data obtained within this research have been stored safely, and there is restricted access to the information to exclude any manipulations.

3.13 Limitations and Threats to Validity

However, the following limitations and validity threats have to be mentioned:

- **Physical Resolution Limits of Hardware:** The hardware precision limit of the used DHT22 sensor equals 0.1°C . It means that any physical changes below this level can not be detected.
- **Localization Bias:** The dataset was gathered in one stable laboratory location. The deployment of our method in the volatile outdoor environment will likely lead to the reduced ratio of the data reduction.
- **Overhead for Asynchronous Re-Alignment:** Since the data is gathered asynchronously, any multi-sensor spatial correlations will need re-alignment and interpolation in the cloud, thus creating some local overhead.

All the limitations and validity threats are analyzed in retrospective and mitigated properly. To avoid validity threats related to statistical conclusion validity, we do some sensitivity analysis of our results by changing slightly the deviation threshold parameters.

Chapter 4

Results and Discussion

4.1 Overview of Collected Environmental Data

Execution of the logging algorithm utilizing the memory-intensive approach during the seven-day monitoring phase led to a significantly reduced amount of data. Assuming that DHT22 sensors were queried every 2 seconds in continuous mode with transmission, the system would produce about 302,400 separate data points. The edge-tier architecture managed to reduce more than 98% of these unnecessary data points using the deviation threshold ($\Delta T \geq 0.5^{\circ}\text{C}$) and time-out ($\Delta t \geq 300\text{s}$) criteria.

The sparse dataset comprised a couple of thousand vital data points. Nonetheless, after analysis of scatter plots from raw data, one can conclude that daily temperature changes, which include slow heating during the daytime and cooling at night-time, were accurately tracked. The highly reduced amount of data translates into direct energy savings in terms of edge nodes' transceivers and cloud storage space for answering RQ1.

4.2 Temperature Trend and Variability Analysis

The descriptive statistics on the filtered dataset gave the initial understanding of the environment under observation. The average temperature within the period of seven days was recorded at 24.3°C , with the minimum temperature of 18.5°C and maximum of 31.2°C . Standard deviation was found to be 2.8°C , thus indicating moderate volatility of the environment, which is quite expected for an indoor environment.

Crucially, statistical comparison between the memorized dataset and a simulated con-

tinuous dataset, created through interpolation, showed insignificant differences in central tendencies (difference in means $< 0.1^{\circ}\text{C}$). It confirms empirically that the memorization rule eliminates only "junk" data and retains the basic statistical characteristics of the physical environment.

4.3 Moving Average and Noise Reduction Effectiveness

Even with the ΔT value, low-cost sensors such as the DHT22 would always have micro-fluctuations at high frequencies that might arise from either drafts or electric noise. In addressing this challenge, the Simple Moving Average method, using a window size of $k = 5$, was employed.

Through the implementation of the SMA method, the time-series was smoothed to give an accurate representation of the curve for the macro changes within the environment. The process helped in minimizing the localized variance without inducing phase shift in the spikes in the temperatures. This indicates that conventional methods can be employed in preparing data for modeling even in asynchronous sampling.

4.4 Temperature Forecasting Using Linear Regression

In order to evaluate the predictive capacity of the compressed data set, the Ordinary Least Squares (OLS) regression model was applied to the temperature values after smoothing. Thus, the resulting equation was obtained:

$$T_{pred}(t) = \beta_0 + \beta_1 t \quad (4.1)$$

Where t is the normalized time elapsed. As a result, a slightly positive value for the slope $\beta_1 > 0$ was found, which reflects a tendency towards global warming on the time interval of 7 days. The high value of the coefficient of determination ($R^2 = 0.81$) of the regression line proves that the memorization method keeps enough data to conduct accurate long-term environmental predictions.

4.5 Anomaly Detection Results

One of the goals was detecting anomalies in the dataset after compression (RQ2). The data points with Z-scores more than 3σ from the mean value were detected using Z-score normalization.

In total, there were 14 anomalies detected in this analysis. More important is that the analysis of timestamps showed that all of the anomalies were related to known artificial disturbances in the environment (for example, the presence of heat source close to the sensor in case of simulated fire or equipment failure). As the memorization algorithm is designed specifically to detect the large deviation and send the notification about it, it becomes an ideal channel for the transmission of anomalies. Conventional duty-cycling methods (such as sleeping for 10 minutes and reading afterwards) may miss the fast 3-minute change in temperature completely. Memorization caught 100% of anomalies instantly.

4.6 Period Segmentation and Change Point Analysis

For RQ3, the Binseg algorithm for offline analysis of structural changes was performed using the data set to detect change points. The algorithm detected change points, thereby dividing the seven-day period into different segments of data that were homogeneous statistically.

The change points were found to be correlated with the daily changes in the environment, including sunrise heating, sunset cooling, and activation/deactivation of the HVAC system.

Table 4.1: Statistical Summary of Period Segmentation via Binseg

Segment	Observed Environmental State / Interpretation
1	Stable nocturnal cooling baseline (18°C - 20°C).
2	Rapid morning heating phase following sunrise.
3	Mid-day volatility (HVAC cycling interference).
4	Evening transition and heat dissipation.

This shows that the use of a non-uniform sampling rate due to memorization does not hinder the use of sophisticated variance-based change point detection technique.

The method has been successful in converting raw and continuous data from sensors to environmental knowledge.

Chapter 5

Conclusion and Future Work

5.1 Summary of the Study

With the exponential rise of the Internet of Things (IoT), there have been huge advancements in environmental sensing and monitoring; however, at the same time, the emergence of such technology has created some serious problems associated with data duplication, network overload, and power usage. In this study, we present and examine a very efficient approach to data reduction at the edge that utilizes the idea of *memorization*. By employing a state-aware algorithm to the Raspberry Pi edge node, our solution prevents the sending of duplicate sensor data unless there was a deviation or time-window.

5.2 Key Findings and Contributions

The findings empirically address all the existing research questions. The algorithm for memorization has effectively filtered over 98% of the initial sensor readings while ensuring the statistical consistency of the dataset (RQ1). It is evident from the use of moving averages and linear regression that macro-level prediction is still quite accurate regardless of the asynchronous nature of the data.

Moreover, the proposed approach was extremely effective for detecting acute anomalies by using Z-score transformation (RQ2) as well as structural environment changes using Binary Segmentation (RQ3). As compared to the interval-based duty-cycling, the approach ensures that the anomaly-triggered memorization mechanism is able to capture

critical events without fail. This study offers an effective approach with low computational cost for bridging the gap between low-power edge processing and high-fidelity cloud processing.

5.3 Practical Implications

This method has a great number of implications related to the creation of scalable IoT architectures. Utilizing the suggested logging technique through memorization allows increasing the life of the devices that work on battery power and other renewable energy sources. Besides, the decrease in the amount of data sent over the network ensures that bandwidth problems do not arise in crowded environments. Cloud computing expenses are going to be minimized.

5.4 Future Research Directions

Although this analysis lays a solid groundwork for the framework to be utilized, several directions for future work can be mentioned:

- **Adaptive Thresholding:** The next versions of the framework should investigate the approach of adaptive thresholding, when the value of ΔT is adjusted dynamically with machine learning in the localized environment depending on its previous volatility.
- **Multi-Variate Memorization:** Enriching the framework to deal with multiple environmental parameters (temperature, humidity, CO₂, VOC, etc.) and detecting cross-correlated anomalies.
- **Hardware Optimization:** Porting the logic implemented in Python into the low-level C/C++ firmware for ultra-low-power microcontrollers (STM32, ESP32, and other) to leverage the energy-efficient properties of the framework at the extreme edge.

References

- [1] H. Sh. Alshehri and F. Bajaber. Improving data aggregation in iot sensor networks using self-organizing maps and firefly optimization algorithm,. H. Sh. Alshehri and F. Bajaber, “Improving data aggregation in IoT sensor networks using self-organizing maps and firefly optimization algorithm,” *Sensors*, vol. 25, no. 23, p. 7107, 2025, doi: 10.3390/s25237107.
- [2] H. Sh. Alshehri and F. Bajaber. Improving data aggregation in iot sensor networks using self-organizing maps and firefly optimization algorithm,. H. Sh. Alshehri and F. Bajaber, “Improving data aggregation in IoT sensor networks using self-organizing maps and firefly optimization algorithm,” *Sensors*, vol. 25, no. 23, p. 7107, 2025, doi: 10.3390/s25237107.
- [3] H. Sh. Alshehri and F. Bajaber. Improving data aggregation in iot sensor networks using self-organizing maps and firefly optimization algorithm,. H. Sh. Alshehri and F. Bajaber, “Improving data aggregation in IoT sensor networks using self-organizing maps and firefly optimization algorithm,” *Sensors*, vol. 25, no. 23, p. 7107, 2025, doi: 10.3390/s25237107.
- [4] E.-S. Apostol, C.-O. Truică, F. Pop, and C. Esposito. Change point enhanced anomaly detection for iot time series data,. E.-S. Apostol, C.-O. Truică, F. Pop, and C. Esposito, “Change point enhanced anomaly detection for IoT time series data,” *Water*, vol. 13, p. 1633, 2021, doi: 10.3390/w13121633.
- [5] E.-S. Apostol, C.-O. Truică, F. Pop, and C. Esposito. Change point enhanced anomaly detection for iot time series data,. E.-S. Apostol, C.-O. Truică, F. Pop, and C. Esposito, “Change point enhanced anomaly detection for IoT time series data,” *Water*, vol. 13, p. 1633, 2021, doi: 10.3390/w13121633.

- [6] E.-S. Apostol, C.-O. Truică, F. Pop, and C. Esposito. Change point enhanced anomaly detection for iot time series data,. E.-S. Apostol, C.-O. Truică, F. Pop, and C. Esposito, “Change point enhanced anomaly detection for IoT time series data,” *Water*, vol. 13, p. 1633, 2021, doi: 10.3390/w13121633.
- [7] A. Bari, S. Wazed, S. Jaekel, and S. Bandyopadhyay. A genetic algorithm-based approach for energy efficient routing in two-tiered sensor networks,. A. Bari, S. Wazed, S. Jaekel, and S. Bandyopadhyay, “A genetic algorithm-based approach for energy efficient routing in two-tiered sensor networks,” *Ad Hoc Networks*, vol. 7, no. 4, pp. 665–676, 2009, doi: 10.1016/j.adhoc.2008.04.003.
- [8] A. Bari, S. Wazed, S. Jaekel, and S. Bandyopadhyay. A genetic algorithm-based approach for energy efficient routing in two-tiered sensor networks,. A. Bari, S. Wazed, S. Jaekel, and S. Bandyopadhyay, “A genetic algorithm-based approach for energy efficient routing in two-tiered sensor networks,” *Ad Hoc Networks*, vol. 7, no. 4, pp. 665–676, 2009, doi: 10.1016/j.adhoc.2008.04.003.
- [9] A. Bari, S. Wazed, S. Jaekel, and S. Bandyopadhyay. A genetic algorithm-based approach for energy efficient routing in two-tiered sensor networks,. A. Bari, S. Wazed, S. Jaekel, and S. Bandyopadhyay, “A genetic algorithm-based approach for energy efficient routing in two-tiered sensor networks,” *Ad Hoc Networks*, vol. 7, no. 4, pp. 665–676, 2009, doi: 10.1016/j.adhoc.2008.04.003.
- [10] J. Barrios-Avilés, A. Rosado-Muñoz, L. D. Medus, M. Bataller-Mompeán, and J. F. Guerrero-Martínez. Less data same information for event-based sensors: A bioinspired filtering and data reduction algorithm,. J. Barrios-Avilés, A. Rosado-Muñoz, L. D. Medus, M. Bataller-Mompeán, and J. F. Guerrero-Martínez, “Less data same information for event-based sensors: A bioinspired filtering and data reduction algorithm,” *Sensors*, vol. 18, no. 12, p. 4122, 2018, doi: 10.3390/s18124122.
- [11] J. Barrios-Avilés, A. Rosado-Muñoz, L. D. Medus, M. Bataller-Mompeán, and J. F. Guerrero-Martínez. Less data same information for event-based sensors: A bioinspired filtering and data reduction algorithm,. J. Barrios-Avilés, A. Rosado-Muñoz, L. D. Medus, M. Bataller-Mompeán, and J. F. Guerrero-Martínez, “Less data same

- information for event-based sensors: A bioinspired filtering and data reduction algorithm,” *Sensors*, vol. 18, no. 12, p. 4122, 2018, doi: 10.3390/s18124122.
- [12] J. Barrios-Avilés, A. Rosado-Muñoz, L. D. Medus, M. Bataller-Mompeán, and J. F. Guerrero-Martínez. Less data same information for event-based sensors: A bioinspired filtering and data reduction algorithm,. J. Barrios-Avilés, A. Rosado-Muñoz, L. D. Medus, M. Bataller-Mompeán, and J. F. Guerrero-Martínez, “Less data same information for event-based sensors: A bioinspired filtering and data reduction algorithm,” *Sensors*, vol. 18, no. 12, p. 4122, 2018, doi: 10.3390/s18124122.
- [13] N. Bui, M. Leng, and M. Rinaldi. N. bui, m. leng, and m. rinaldi, 2020 – hybrid iot architecture for smart metering: Integration of lorawan and wifi for advanced meter reading applications. N. Bui, M. Leng, and M. Rinaldi, 2020 – Hybrid IoT architecture for smart metering: Integration of LoRaWAN and WiFi for advanced meter reading applications.
- [14] N. Bui, M. Leng, and M. Rinaldi. N. bui, m. leng, and m. rinaldi, 2020 – hybrid iot architecture for smart metering: Integration of lorawan and wifi for advanced meter reading applications. N. Bui, M. Leng, and M. Rinaldi, 2020 – Hybrid IoT architecture for smart metering: Integration of LoRaWAN and WiFi for advanced meter reading applications.
- [15] N. Bui, M. Leng, and M. Rinaldi. N. bui, m. leng, and m. rinaldi, 2020 – hybrid iot architecture for smart metering: Integration of lorawan and wifi for advanced meter reading applications. N. Bui, M. Leng, and M. Rinaldi, 2020 – Hybrid IoT architecture for smart metering: Integration of LoRaWAN and WiFi for advanced meter reading applications.
- [16] S. Deldari, D. V. Smith, H. Xue, and F. D. Salim. Time series change point detection with self-supervised contrastive predictive coding,. S. Deldari, D. V. Smith, H. Xue, and F. D. Salim, “Time series change point detection with self-supervised contrastive predictive coding,” in *Proceedings of the Web Conference 2021 (WWW ’21)*, New York, NY, USA, 2021, pp. 3124–3135, doi: 10.1145/3442381.3449903.
- [17] S. Deldari, D. V. Smith, H. Xue, and F. D. Salim. Time series change point detection with self-supervised contrastive predictive coding,. S. Deldari, D. V. Smith, H. Xue,

- and F. D. Salim, “Time series change point detection with self-supervised contrastive predictive coding,” in *Proceedings of the Web Conference 2021 (WWW ’21)*, New York, NY, USA, 2021, pp. 3124–3135, doi: 10.1145/3442381.3449903.
- [18] S. Deldari, D. V. Smith, H. Xue, and F. D. Salim. Time series change point detection with self-supervised contrastive predictive coding,. S. Deldari, D. V. Smith, H. Xue, and F. D. Salim, “Time series change point detection with self-supervised contrastive predictive coding,” in *Proceedings of the Web Conference 2021 (WWW ’21)*, New York, NY, USA, 2021, pp. 3124–3135, doi: 10.1145/3442381.3449903.
- [19] X. Du, Z. Zhou, Y. Zhang, T. Rahman, et al. Energy-efficient sensory data gathering based on compressed sensing in iot networks,. X. Du, Z. Zhou, Y. Zhang, T. Rahman, et al., “Energy-efficient sensory data gathering based on compressed sensing in IoT networks,” *Journal of Cloud Computing: Advances, Systems and Applications*, vol. 9, article 19, 2020, doi: 10.1186/s13677-020-00166-x.
- [20] X. Du, Z. Zhou, Y. Zhang, T. Rahman, et al. Energy-efficient sensory data gathering based on compressed sensing in iot networks,. X. Du, Z. Zhou, Y. Zhang, T. Rahman, et al., “Energy-efficient sensory data gathering based on compressed sensing in IoT networks,” *Journal of Cloud Computing: Advances, Systems and Applications*, vol. 9, article 19, 2020, doi: 10.1186/s13677-020-00166-x.
- [21] X. Du, Z. Zhou, Y. Zhang, T. Rahman, et al. Energy-efficient sensory data gathering based on compressed sensing in iot networks,. X. Du, Z. Zhou, Y. Zhang, T. Rahman, et al., “Energy-efficient sensory data gathering based on compressed sensing in IoT networks,” *Journal of Cloud Computing: Advances, Systems and Applications*, vol. 9, article 19, 2020, doi: 10.1186/s13677-020-00166-x.
- [22] J. Elson, L. Girod, and D. Estrin. Fine-grained network time synchronization using reference broadcasts,. J. Elson, L. Girod, and D. Estrin, “Fine-grained network time synchronization using reference broadcasts,” *ACM SIGOPS Operating Systems Review*, vol. 36, no. SI, pp. 147–163, Dec. 2002, doi: 10.1145/844128.844143.
- [23] J. Elson, L. Girod, and D. Estrin. Fine-grained network time synchronization using reference broadcasts,. J. Elson, L. Girod, and D. Estrin, “Fine-grained network

- time synchronization using reference broadcasts,” *ACM SIGOPS Operating Systems Review*, vol. 36, no. SI, pp. 147–163, Dec. 2002, doi: 10.1145/844128.844143.
- [24] J. Elson, L. Girod, and D. Estrin. Fine-grained network time synchronization using reference broadcasts,. J. Elson, L. Girod, and D. Estrin, “Fine-grained network time synchronization using reference broadcasts,” *ACM SIGOPS Operating Systems Review*, vol. 36, no. SI, pp. 147–163, Dec. 2002, doi: 10.1145/844128.844143.
- [25] L. Erhan, M. Ndubuaku, M. Di Mauro, W. Song, M. Chen, G. Fortino, O. Bagdasar, and A. Liotta. Smart anomaly detection in sensor systems: A multi-perspective review,. L. Erhan, M. Ndubuaku, M. Di Mauro, W. Song, M. Chen, G. Fortino, O. Bagdasar, and A. Liotta, “Smart anomaly detection in sensor systems: A multi-perspective review,” *Information Fusion*, vol. 67, pp. 64–79, 2020, doi: 10.1016/j.inffus.2020.10.001.
- [26] L. Erhan, M. Ndubuaku, M. Di Mauro, W. Song, M. Chen, G. Fortino, O. Bagdasar, and A. Liotta. Smart anomaly detection in sensor systems: A multi-perspective review,. L. Erhan, M. Ndubuaku, M. Di Mauro, W. Song, M. Chen, G. Fortino, O. Bagdasar, and A. Liotta, “Smart anomaly detection in sensor systems: A multi-perspective review,” *Information Fusion*, vol. 67, pp. 64–79, 2020, doi: 10.1016/j.inffus.2020.10.001.
- [27] L. Erhan, M. Ndubuaku, M. Di Mauro, W. Song, M. Chen, G. Fortino, O. Bagdasar, and A. Liotta. Smart anomaly detection in sensor systems: A multi-perspective review,. L. Erhan, M. Ndubuaku, M. Di Mauro, W. Song, M. Chen, G. Fortino, O. Bagdasar, and A. Liotta, “Smart anomaly detection in sensor systems: A multi-perspective review,” *Information Fusion*, vol. 67, pp. 64–79, 2020, doi: 10.1016/j.inffus.2020.10.001.
- [28] A. Fascista, A. Coluccia, and C. Ravazzi. A unified bayesian framework for joint estimation and anomaly detection in environmental sensor networks,. A. Fascista, A. Coluccia and C. Ravazzi, ”A Unified Bayesian Framework for Joint Estimation and Anomaly Detection in Environmental Sensor Networks,” in *IEEE Access*, vol. 11, pp. 227-248, 2023, doi: 10.1109/ACCESS.2022.3231909.

- [29] A. Fascista, A. Coluccia, and C. Ravazzi. A unified bayesian framework for joint estimation and anomaly detection in environmental sensor networks,. A. Fascista, A. Coluccia and C. Ravazzi, "A Unified Bayesian Framework for Joint Estimation and Anomaly Detection in Environmental Sensor Networks," in IEEE Access, vol. 11, pp. 227-248, 2023, doi: 10.1109/ACCESS.2022.3231909.
- [30] A. Fascista, A. Coluccia, and C. Ravazzi. A unified bayesian framework for joint estimation and anomaly detection in environmental sensor networks,. A. Fascista, A. Coluccia and C. Ravazzi, "A Unified Bayesian Framework for Joint Estimation and Anomaly Detection in Environmental Sensor Networks," in IEEE Access, vol. 11, pp. 227-248, 2023, doi: 10.1109/ACCESS.2022.3231909.
- [31] S. Kumar and P. O. Sarangamath. Smart environmental monitoring system using raspberry pi and dht11 sensor,. S. Kumar and P. O. Sarangamath, "Smart Environmental Monitoring System Using Raspberry PI and DHT11 Sensor," International Journal for Research in Applied Science & Engineering Technology (IJRASET), vol. 13, no. IX, Sep. 2025, doi: 10.22214/ijraset.2025.74184.
- [32] S. Kumar and P. O. Sarangamath. Smart environmental monitoring system using raspberry pi and dht11 sensor,. S. Kumar and P. O. Sarangamath, "Smart Environmental Monitoring System Using Raspberry PI and DHT11 Sensor," International Journal for Research in Applied Science & Engineering Technology (IJRASET), vol. 13, no. IX, Sep. 2025, doi: 10.22214/ijraset.2025.74184.
- [33] S. Kumar and P. O. Sarangamath. Smart environmental monitoring system using raspberry pi and dht11 sensor,. S. Kumar and P. O. Sarangamath, "Smart Environmental Monitoring System Using Raspberry PI and DHT11 Sensor," International Journal for Research in Applied Science & Engineering Technology (IJRASET), vol. 13, no. IX, Sep. 2025, doi: 10.22214/ijraset.2025.74184.
- [34] K. Lee, S. Jeong, S. H. Sim, and D. H. Shin. A novelty detection approach for tendons of prestressed concrete bridges based on a convolutional autoencoder and acceleration data,. K. Lee, S. Jeong, S. H. Sim, and D. H. Shin, "A novelty detection approach for tendons of prestressed concrete bridges based on a convolutional autoencoder and acceleration data," Sensors, vol. 19, no. 7, p. 1633, 2019, doi: 10.3390/s19071633.

- [35] K. Lee, S. Jeong, S. H. Sim, and D. H. Shin. A novelty detection approach for tendons of prestressed concrete bridges based on a convolutional autoencoder and acceleration data,. K. Lee, S. Jeong, S. H. Sim, and D. H. Shin, “A novelty detection approach for tendons of prestressed concrete bridges based on a convolutional autoencoder and acceleration data,” *Sensors*, vol. 19, no. 7, p. 1633, 2019, doi: 10.3390/s19071633.
- [36] K. Lee, S. Jeong, S. H. Sim, and D. H. Shin. A novelty detection approach for tendons of prestressed concrete bridges based on a convolutional autoencoder and acceleration data,. K. Lee, S. Jeong, S. H. Sim, and D. H. Shin, “A novelty detection approach for tendons of prestressed concrete bridges based on a convolutional autoencoder and acceleration data,” *Sensors*, vol. 19, no. 7, p. 1633, 2019, doi: 10.3390/s19071633.
- [37] J. Li, Z. Li, H. Wu, and N. You. Trend, seasonality, and abrupt change detection method for land surface temperature time-series analysis: Evaluation and improvement,. J. Li, Z. Li, H. Wu, and N. You, “Trend, seasonality, and abrupt change detection method for land surface temperature time-series analysis: Evaluation and improvement,” *Remote Sensing of Environment*, vol. 280, p. 113222, 2022, doi: 10.1016/j.rse.2022.113222.
- [38] J. Li, Z. Li, H. Wu, and N. You. Trend, seasonality, and abrupt change detection method for land surface temperature time-series analysis: Evaluation and improvement,. J. Li, Z. Li, H. Wu, and N. You, “Trend, seasonality, and abrupt change detection method for land surface temperature time-series analysis: Evaluation and improvement,” *Remote Sensing of Environment*, vol. 280, p. 113222, 2022, doi: 10.1016/j.rse.2022.113222.
- [39] J. Li, Z. Li, H. Wu, and N. You. Trend, seasonality, and abrupt change detection method for land surface temperature time-series analysis: Evaluation and improvement,. J. Li, Z. Li, H. Wu, and N. You, “Trend, seasonality, and abrupt change detection method for land surface temperature time-series analysis: Evaluation and improvement,” *Remote Sensing of Environment*, vol. 280, p. 113222, 2022, doi: 10.1016/j.rse.2022.113222.
- [40] P. Mignone, R. Corizzo, and M. Ceci. Distributed and explainable ghsom for anomaly detection in sensor networks,. P. Mignone, R. Corizzo, and M. Ceci, “Distributed and

- explainable GHSOM for anomaly detection in sensor networks,” *Machine Learning*, vol. 113, pp. 4445–4486, 2024, doi: 10.1007/s10994-023-06501-y.
- [41] P. Mignone, R. Corizzo, and M. Ceci. Distributed and explainable ghsom for anomaly detection in sensor networks,. P. Mignone, R. Corizzo, and M. Ceci, “Distributed and explainable GHSOM for anomaly detection in sensor networks,” *Machine Learning*, vol. 113, pp. 4445–4486, 2024, doi: 10.1007/s10994-023-06501-y.
- [42] P. Mignone, R. Corizzo, and M. Ceci. Distributed and explainable ghsom for anomaly detection in sensor networks,. P. Mignone, R. Corizzo, and M. Ceci, “Distributed and explainable GHSOM for anomaly detection in sensor networks,” *Machine Learning*, vol. 113, pp. 4445–4486, 2024, doi: 10.1007/s10994-023-06501-y.
- [43] A. F. Militino, M. Moradi, and M. D. Ugarte. On the performances of trend and change-point detection methods for remote sensing data,. A. F. Militino, M. Moradi, and M. D. Ugarte, “On the performances of trend and change-point detection methods for remote sensing data,” *Remote Sensing*, vol. 12, p. 1008, 2020, doi: 10.3390/rs12061008.
- [44] A. F. Militino, M. Moradi, and M. D. Ugarte. On the performances of trend and change-point detection methods for remote sensing data,. A. F. Militino, M. Moradi, and M. D. Ugarte, “On the performances of trend and change-point detection methods for remote sensing data,” *Remote Sensing*, vol. 12, p. 1008, 2020, doi: 10.3390/rs12061008.
- [45] A. F. Militino, M. Moradi, and M. D. Ugarte. On the performances of trend and change-point detection methods for remote sensing data,. A. F. Militino, M. Moradi, and M. D. Ugarte, “On the performances of trend and change-point detection methods for remote sensing data,” *Remote Sensing*, vol. 12, p. 1008, 2020, doi: 10.3390/rs12061008.
- [46] S. Polymeni, E. Athanasakis, G. Spanos, K. Votis, and D. Tzovaras. Iot-based prediction models in the environmental context: A systematic literature review,. S. Polymeni, E. Athanasakis, G. Spanos, K. Votis, and D. Tzovaras, “IoT-based prediction models in the environmental context: A systematic literature review,” *Internet of Things*, vol. 20, p. 100612, 2022, doi: 10.1016/j.iot.2022.100612.

- [47] S. Polymeni, E. Athanasakis, G. Spanos, K. Votis, and D. Tzovaras. Iot-based prediction models in the environmental context: A systematic literature review,. S. Polymeni, E. Athanasakis, G. Spanos, K. Votis, and D. Tzovaras, “IoT-based prediction models in the environmental context: A systematic literature review,” *Internet of Things*, vol. 20, p. 100612, 2022, doi: 10.1016/j.iot.2022.100612.
- [48] S. Polymeni, E. Athanasakis, G. Spanos, K. Votis, and D. Tzovaras. Iot-based prediction models in the environmental context: A systematic literature review,. S. Polymeni, E. Athanasakis, G. Spanos, K. Votis, and D. Tzovaras, “IoT-based prediction models in the environmental context: A systematic literature review,” *Internet of Things*, vol. 20, p. 100612, 2022, doi: 10.1016/j.iot.2022.100612.
- [49] S. Russo, M. D. Lürig, W. Hao, B. Matthews, and K. Villez. Active learning for anomaly detection in environmental data,. S. Russo, M. D. Lürig, W. Hao, B. Matthews, and K. Villez, “Active learning for anomaly detection in environmental data,” *Environmental Modelling and Software*, vol. 134, p. 104869, Dec. 2020, doi: 10.1016/j.envsoft.2020.104869.
- [50] S. Russo, M. D. Lürig, W. Hao, B. Matthews, and K. Villez. Active learning for anomaly detection in environmental data,. S. Russo, M. D. Lürig, W. Hao, B. Matthews, and K. Villez, “Active learning for anomaly detection in environmental data,” *Environmental Modelling and Software*, vol. 134, p. 104869, Dec. 2020, doi: 10.1016/j.envsoft.2020.104869.
- [51] S. Russo, M. D. Lürig, W. Hao, B. Matthews, and K. Villez. Active learning for anomaly detection in environmental data,. S. Russo, M. D. Lürig, W. Hao, B. Matthews, and K. Villez, “Active learning for anomaly detection in environmental data,” *Environmental Modelling and Software*, vol. 134, p. 104869, Dec. 2020, doi: 10.1016/j.envsoft.2020.104869.
- [52] H. Y. Teh, A. W. Kempa-Liehr, and K. I. K. Wang. Sensor data quality: A systematic review,. H. Y. Teh, A. W. Kempa-Liehr, and K. I. K. Wang, “Sensor data quality: A systematic review,” *Journal of Big Data*, vol. 7, no. 11, 2020, doi: 10.1186/s40537-020-0285-1.

- [53] H. Y. Teh, A. W. Kempa-Liehr, and K. I. K. Wang. Sensor data quality: A systematic review,. H. Y. Teh, A. W. Kempa-Liehr, and K. I. K. Wang, “Sensor data quality: A systematic review,” *Journal of Big Data*, vol. 7, no. 11, 2020, doi: 10.1186/s40537-020-0285-1.
- [54] H. Y. Teh, A. W. Kempa-Liehr, and K. I. K. Wang. Sensor data quality: A systematic review,. H. Y. Teh, A. W. Kempa-Liehr, and K. I. K. Wang, “Sensor data quality: A systematic review,” *Journal of Big Data*, vol. 7, no. 11, 2020, doi: 10.1186/s40537-020-0285-1.
- [55] C. Truong, L. Oudre, and N. Vayatis. Selective review of offline change point detection methods,. C. Truong, L. Oudre, and N. Vayatis, “Selective review of offline change point detection methods,” *Signal Processing*, vol. 167, p. 107299, 2020, doi: 10.1016/j.sigpro.2019.107299.
- [56] C. Truong, L. Oudre, and N. Vayatis. Selective review of offline change point detection methods,. C. Truong, L. Oudre, and N. Vayatis, “Selective review of offline change point detection methods,” *Signal Processing*, vol. 167, p. 107299, 2020, doi: 10.1016/j.sigpro.2019.107299.
- [57] C. Truong, L. Oudre, and N. Vayatis. Selective review of offline change point detection methods,. C. Truong, L. Oudre, and N. Vayatis, “Selective review of offline change point detection methods,” *Signal Processing*, vol. 167, p. 107299, 2020, doi: 10.1016/j.sigpro.2019.107299.
- [58] H. Zhang, J. Na, and B. Zhang. Autonomous internet of things (iot) data reduction based on adaptive threshold,. H. Zhang, J. Na, and B. Zhang, “Autonomous Internet of Things (IoT) data reduction based on adaptive threshold,” *Sensors*, vol. 23, no. 23, p. 9427, 2023, doi: 10.3390/s23239427.
- [59] H. Zhang, J. Na, and B. Zhang. Autonomous internet of things (iot) data reduction based on adaptive threshold,. H. Zhang, J. Na, and B. Zhang, “Autonomous Internet of Things (IoT) data reduction based on adaptive threshold,” *Sensors*, vol. 23, no. 23, p. 9427, 2023, doi: 10.3390/s23239427.
- [60] H. Zhang, J. Na, and B. Zhang. Autonomous internet of things (iot) data reduction based on adaptive threshold,. H. Zhang, J. Na, and B. Zhang, “Autonomous Internet

of Things (IoT) data reduction based on adaptive threshold,” *Sensors*, vol. 23, no. 23, p. 9427, 2023, doi: 10.3390/s23239427.